# WOJSKOWA AKADEMIA TECHNICZNA
**im. Jarosława Dąbrowskiego**

## WYDZIAŁ CYBERNETYKI





# PRACA DYPLOMOWA
STUDIA I$^{o}$

Temat pracy: **Środowisko CI/CD dla modeli uczenia maszynowego**

**INFORMATYKA**

(kierunek studiów)

**INŻYNIERIA SYSTEMÓW - ANALIZA DANYCH**

(specjalność)


Dyplomant: **Mateusz SZYMCZYK**

Promotor: **mgr inż. Joanna WOŹNIAK**



**Warszawa 2024**

**OŚWIADCZENIE**

*Wyrażam zgodę / ~~nie wyrażam zgody~~* *
*na udostępnianie mojej pracy przez Archiwum WAT*

Dnia ..... .....

(podpis)

* *Niepotrzebne skreślić*

# Spis treści

Wstęp ..... 4
Rozdział I. Podstawy teoretyczne ..... 5
I.1. Uczenie Maszynowe ..... 5
I.2. CI/CD ..... 5
I.3. Cylk życia modelu uczenia maszynowego ..... 6
Rozdział II. Przegląd istniejących rozwiązań ..... 9
II.1. Wymagania dotyczące platformy MLOps ..... 9
II.2. Wybrane pełnoprawne otwartoźródłowe platformy MLOps ..... 9
II.3. Uzasadnienie wyboru orkiestratora ..... 13
Rozdział III. Kubeflow ..... 15
III.1. Komponenty Kubeflow ..... 15
Rozdział IV. Implementacja ..... 23
IV.1. Wprowadzenie ..... 23
IV.2. Instalacja Kubeflow ..... 30
IV.3. Testy akceptacyjne ..... 32
IV.4. CI/CD w Kubeflow ..... 34
IV.5. Implementacja Mikrousługi ..... 40
Podsumowanie ..... 42
Bibliografia ..... 43
Spis rysunków ..... 46
Spis tabel ..... 46
Załączniki ..... 47

## Wstęp

Niniejsza praca dyplomowa przedstawia implementację środowiska Ciągłej integracji-/Ciągłego wdrażania (z ang. Continuous Integration/Continuous Delivery, dalej CI/CD) dla modeli uczenia maszynowego. W dobie dynamicznie rozwijającej się dziedziny uczenia maszynowego oraz wzrastającej roli automatyzacji procesów deweloperskich, zapewnienie efektywnego i niezawodnego środowiska CI/CD staje się kluczowe dla osiągnięcia sukcesu w projektach związanych z uczeniem maszynowym (z ang. Machine Learning, dalej ML). Praca ta koncentruje się na wykorzystaniu platformy Kubeflow, integrując ją z popularnymi narzędziami i technikami wykorzystywanymi w CI/CD, co stanowi odpowiedź na rosnące wymagania rynku w zakresie szybkiego i skutecznego wdrażania modeli uczenia maszynowego. Struktura pracy obejmuje następujące rozdziały:

1. Podstawy teoretyczne: Omówienie podstawowych pojęć związanych z uczeniem maszynowym oraz CI/CD, a także analiza cyklu życia modelu uczenia maszynowego.

2. Przegląd istniejących rozwiązań: Prezentacja wymagań dla platform MLOps i dyskusja na temat wybranych platform open-source.

3. Kubeflow: Szczegółowe wprowadzenie do platformy Kubeflow, z analizą jej komponentów i powodów wykorzystania Kubernetes.

4. Implementacja: Praktyczna realizacja środowiska CI/CD w Kubeflow oraz mikrousługi, wraz z instrukcjami dotyczącymi instalacji i konfiguracji.

# Rozdział I. Podstawy teoretyczne

## I.1. Uczenie Maszynowe

Autorzy [1] definiują uczenie maszynowe jako obszar badań, który koncentruje się na tym, jak komputer może "uczyć się"wykonywania określonych zadań, takich jak rozpoznawanie znaków, wspomaganie diagnozy chorób, klasyfikacja typów win, segregacja materiałów według ich jakości, a także wielu innych zastosowań. Uczenie maszynowe ma na celu rozwiązywanie codziennych problemów i wspieranie specjalistów w podejmowaniu decyzji. Jest to obszar, który przyciąga badaczy z różnych dziedzin wiedzy.

Podstawą uczenia maszynowego jest przetwarzanie i analiza dużych ilości danych. Proces ten wymaga zrozumienia i właściwej interpretacji tych zbiorów. Ma to kluczowe znaczenie dla skutecznego nauczania i wdrażania modeli. Wykorzystuje się algorytmy, które uczą się z dostępnych materiałów i mogą na przykład przewidywać wyniki lub identyfikować wzorce, które nie są oczywiste dla ludzkiego oka.

Równie ważne jest zrozumienie, że uczenie maszynowe wymaga ciągłego dostosowywania i ulepszania. Modele muszą być regularnie aktualizowane w celu zachowania ich dokładności i efektywności, zwłaszcza w dynamicznie zmieniającym się świecie danych. Dlatego proces ten nie kończy się na pierwszym wdrożeniu modelu. Jest to ciągły cykl rozwoju, testowania, oceny i optymalizacji.

Takie podejście podkreśla, że uczenie maszynowe nie jest statyczną dziedziną, lecz ewoluującym procesem, który wymaga adaptacji do nowych wyzwań i danych. Dzięki temu, uczenie maszynowe staje się nie tylko narzędziem technologicznym, ale także katalizatorem innowacji w różnych dziedzinach przemysłu i nauki.

## I.2. CI/CD

Autor [3] określa CI/CD jako zestaw praktyk i metodologii mających na celu zautomatyzowanie procesu dostarczania oprogramowania. Proces ten obejmuje zarówno integrację ciągłą, jak i ciągłą dostawę, stanowiąc fundament dla szybkiego, bezpiecznego i efektywnego wdrażania kodu aplikacji do produkcji.

- **Integracja Ciągła (z angielskiego Continous Integration, dalej CI)** jest to proces, w którym deweloperzy regularnie (często kilka razy dziennie) integrują swoje zmiany kodu do głównego repozytorium projektu. Głównym celem CI jest szybka identyfikacja i naprawa problemów wynikających z integracji, co pozwala na utrzymanie stabilności kodu i zapewnienie ciągłej gotowości do produkcji. W tym podejściu, zaraz po przesłaniu zmian, kod jest automatycznie budowany i poddawany szeregowi testów (jednostkowych, integracyjnych, funkcjonalnych itp.), co pozwala na wychwycenie błędów na wczesnym etapie. CI zwiększa również przejrzystość pracy w zespole, ponieważ wszyscy członkowie mają dostęp do najnowszej wersji kodu i widzą efekty wprowadzanych zmian.pozytorium. Proces ten jest powtarzalny, co zapewnia konsystencję i szybkość wykonania.

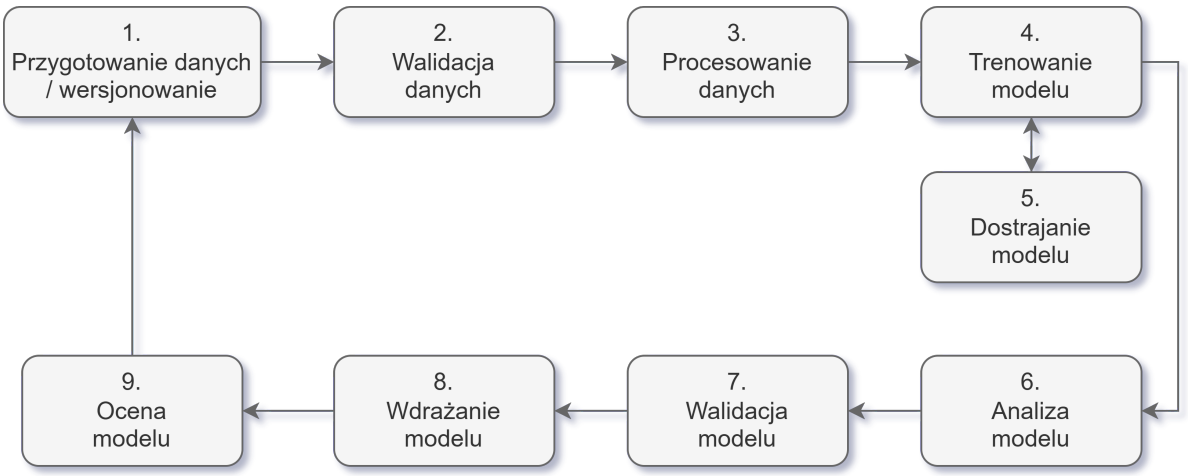
- **Ciągła Dostawa (z angielskiego Continous Delivery, dalej CD)** jest rozszerzeniem Integracji ciągłej, polegającym na automatycznym wdrażaniu każdej zmiany

kodu, która pomyślnie przeszła przez proces CI, bezpośrednio do środowiska produkcyjnego. CD eliminuje potrzebę ręcznego wdrażania oprogramowania, co znacznie przyspiesza proces dostarczania nowych funkcji i poprawek do użytkowników końcowych. Jest to możliwe dzięki zautomatyzowanym procesom, które sprawdzają jakość kodu, testują go i wdrażają, zapewniając ciągłość i stabilność wersji produkcyjnej. Zastosowanie CD wymaga jednak wysokiego poziomu zaufania do procesów testowania i monitorowania, ponieważ każda zmiana może być natychmiastowo widoczna dla użytkowników.

Podsumowując, CI/CD jest złożonym zestawem praktyk, które wymagają holistycznego podejścia do całego procesu tworzenia oprogramowania. Od analizy wymagań po wdrożenie i monitorowanie, każdy etap ma kluczowe znaczenie dla sukcesu w automatyzacji dostarczania oprogramowania. Dzięki temu, organizacje mogą szybko dostosowywać się do zmieniających się wymagań rynku i technologii, jednocześnie utrzymując wysoką jakość i bezpieczeństwo swoich produktów.

## I.3. Cylk życia modelu uczenia maszynowego

Autorzy [3] określają cykl życia modelu uczenia maszynowego jako wszystkie etapy związane z tworzeniem, wdrażaniem i utrzymaniem modeli. W zależności od projektu, niektóre etapy mogą być pomijane lub powtarzane wielokrotnie. Cykl życia modelu uczenia maszynowego przedstawiony na rysunku 1 można podzielić na dziewięć głównych etapów:




**Rys. 1. Cykl życia modelu uczenia maszynowego**

Źródło: Badania własne.

1. **Przygotowanie danych/wersjonowanie.** Pozyskiwanie danych, jest początkiem każdego potoku uczenia maszynowego. Na tym etapie potoku przetwarzamy dane do formatu, który może zostać przetworzony przez kolejne komponenty. Jest to

również odpowiedni moment na wersjonowanie przychodzących danych, umożliwiając powiązanie danej wersji danych ze szkolonym modelem.

2. **Walidacja Danych.** Przed szkoleniem nowej wersji modelu konieczna jest walidacja nowych danych. Skupia się ona na sprawdzeniu, czy statystyki nowych danych odpowiadają oczekiwaniom, np. zakresu, liczby kategorii czy rozkładu kategorii. Narzędzia walidacji danych alarmują również, gdy wykryją anomalie. Jeśli walidacja ujawni coś niezwykłego, proces może zostać wstrzymany, a analityk danych poinformowany. W takiej sytuacji możliwe jest dostosowanie próbkowania klas lub zmiana funkcji straty modelu, uruchomienie nowego procesu budowy modelu i ponowne rozpoczęcie cyklu życia.

3. **Wstępne przetwarzanie danych.** Jest wysoce prawdopodobne, że nie można użyć świeżo zebranych danych i bezpośrednio trenować modelu uczenia maszynowego. W prawie wszystkich przypadkach konieczne będzie wstępne przetworzenie danych w celu wykorzystania ich do treningów. Etykiety często muszą zostać przekonwertowane na jeden lub wiele wektorów. To samo dotyczy danych wejściowych modelu. Przy trenowaniu modelu z danych tekstowych, trzeba przekonwertować znaki tekstu na indeksy lub tokeny tekstowe na wektory słów.

4. **Trenowanie Modelu.** Jest to kluczowy etap, podczas którego model uczy się na podstawie dostarczonych danych, dopasowując swoje wagi i parametry w celu minimalizacji błędu w przewidywaniach.

5. **Dostrajane modelu.** Dostrajanie modelu polega na optymalizacji jego parametrów w celu uzyskania najlepszych możliwych wyników. Proces ten może obejmować modyfikację hiperparametrów, takich jak szybkość uczenia się, aby dopasować model do specyfikacji problemu i zwiększyć jego dokładność.

6. **Analiza Modelu**. Analiza modelu skupia się na ocenie jego wydajności i przydatności w rozwiązywaniu określonych problemów. W tym etapie analizuje się różne metryki, takie jak dokładność, precyzja i czułość, aby ocenić, jak dobrze model radzi sobie z zadaniem.

7. **Walidacja i Testowanie Modelu.** Walidacja i testowanie modelu to etapy, na których oceniana jest jego wydajność, zapewniając niezawodność i skuteczność modelu w praktycznym zastosowaniu. Testowanie na niezależnych zestawach danych pozwala na weryfikację, czy model zachowuje spodziewaną wydajność i unika przeuczenia.

8. **Wdrażanie Modelu.** Wdrażanie modelu to proces umieszczania go w środowisku produkcyjnym, gdzie może być używany do przewidywań w rzeczywistych scenariuszach. W tym etapie model zostaje zintegrowany z systemami biznesowymi, co pozwala na wykorzystanie jego zdolności predykcyjnych w praktyce.

9. **Monitorowanie i Utrzymanie.** Stałe śledzenie wydajności modelu, dostosowywanie i aktualizacja w razie potrzeby.

## Integracja CI/CD z Cyklem Życia Modelu

Włączenie koncepcji CI/CD w cykl życia modelu machine learning umożliwia automatyzację i ulepszenie procesu tworzenia modeli. Ułatwia to szybkie wdrażanie zmian, testowanie modeli w kontrolowanych warunkach i zapewnia ciągłą poprawę wydajności modeli. Autorzy[3] podkreślają, jak integracja CI/CD z cyklem życia modelu przyczynia się do zwiększenia efektywności, redukcji błędów i poprawy jakości modeli uczenia maszynowego.

# Rozdział II. Przegląd istniejących rozwiązań

## II.1. Wymagania dotyczące platformy MLOps

Niniejsza praca dyplomowa koncentruje się na wykorzystaniu rozwiązań open-source (otwartożródłowe), z uwagi na szereg istotnych zalet, które te rozwiązania oferują. Przede wszystkim, takie oprogramowanie zapewnia większą elastyczność i możliwości dostosowania do specyficznych wymagań projektu. Dostępność kodu źródłowego umożliwia dokładne zrozumienie działania narzędzi i adaptację ich do konkretnych potrzeb. Ponadto, społeczności open source często są dynamiczne i innowacyjne, co sprzyja szybkiemu rozwojowi i ulepszaniu narzędzi.

Kolejnym ważnym aspektem jest aspekt kosztowy - oprogramowanie open-source zazwyczaj jest bezpłatne, co jest istotne zwłaszcza w kontekście projektów akademickich lub startupów z ograniczonym budżetem. Dodatkowo, korzystanie z otwartych rozwiązań sprzyja edukacji i wymianie wiedzy, co jest szczególnie ważne w środowisku akademickim.

W ramach pracy dyplomowej brane są pod uwagę wyłącznie na kompleksowe platformy MLOps (ang. Machine Learning Operations, czyli Operacje Uczenia Maszynowego). Są to platformy zawierające narzędzia potrzebne na wszystkich etapach przepływu pracy związanego z uczeniem maszynowym. Idealnie, korzystając z pełnoprawnej platformy, nie powinno być konieczności konfigurowania dodatkowych narzędzi. W praktyce jednak, zależy to od wymagań danego projektu oraz preferencji użytkownika.

## II.2. Wybrane pełnoprawne otwartoźródłowe platformy MLOps

Isnieje wiele konkurujących ze sobą platform. Każda z nich ma swoje cechy charaktrystyczne oraz mocniejsze strony. Wybór odpowiedniej jest kluczową decyzją każdego projektu. Dlatego bardzo ważne jest regularne badanie rynku, aby znać najnowocześniejsze narzędzia. Odpowiednio wybrane i zaprojektowana platforma jest w stanie zaoszczędzić bardzo dużo czasu, a co za tym idzie pieniędzy. Przykładami tego są takie platformy jak:

### Kubeflow[5]

Prawie natychmiast po tym, jak Kubernetes stał się jednym z głównych standardów do pracy z grupą kontenerów, Google stworzyło Kubeflow - projekt open-source, który upraszcza pracę z ML w Kubernetes. Posiada wszystkie zalety tego narzędzia do orkiestracji, od możliwości wdrożenia na dowolnej infrastrukturze po zarządzanie luźno powiązanymi mikrousługami i skalowanie na żądanie. Ten projekt jest dla programistów, którzy chcą wdrażać przenośne i skalowalne projekty uczenia maszynowego. Może być stosowany w różnych infrastrukturach - od superkomputerów po laptopy. Vertex AI wykorzystuje Kubeflow jako podstawową platformę do tworzenia swoich Pipelines. Pokazuje to, jak platforma open-source może być rozszerzana w ramach komercyjnych produktów.

Kubeflow wyróżnia się następującymi cechami:

- **Notatniki Jupyter.** Tworzenie i dostosowywanie notatników Jupyter, natychmiastowe widzenie wyników uruchamianego kodu i tworzenie interaktywnych raportów analitycznych.
- **Operatory trenowania.** Kubeflow training operator to zasób niestandardowy Kubernetes, który ułatwia uruchamianie skalowalnych i rozproszonych zadań treningowych dla popularnych frameworków, takich jak TensorFlow, PyTorch, MPI, MXNet, PaddlePaddle i XGBoost. Jego zaletą jest to, że zapewnia jednolity interfejs dla różnych modeli treningowych na Kubernetes.
- **Uproszczona konteneryzacja.** Kubeflow eliminuje złożoność związaną z konteneryzacją kodu. Naukowcy zajmujący się danymi mogą wykonywać przygotowanie danych, szkolenie modeli i wdrażanie w krótszym czasie.

Podsumowując, Kubeflow to kompleksowe rozwiązanie do tworzenia i wdrażania end-to-end przepływów pracy związanych z uczeniem maszynowym.

### MLflow[9]

MLflow to stworzona przez Databricks platforma open-source dla inżynierów uczenia maszynowego do zarządzania cyklem życia uczenia maszynowego poprzez eksperymentowanie, wdrażanie i testowanie. MLflow jest przydatny do śledzenia wydajność modeli uczenia maszynowego. Platforma ta stanowi swoistego rodzaju centrum dowodzenia, które pozwala:

- monitorować przepływy pracy uczenia maszynowego,
- przechowywać metadane modelu,
- wybierać najlepiej działający model.

**Sklada się z trzech głównych komponentów:**

- **MLflow Tracking.** API i interfejs użytkownika do rejestrowania parametrów, wersji kodu, metryk i plików wyjściowych dla uruchamianego kodu i wizualizacji wyników.
- **MLflow Project.** Narzędzie dla zespołów zajmujących się uczeniem maszynowym do pakowania kodu naukowego danych w sposób możliwy do ponownego użycia i powtarzalny.
- **MLflow Model.** ułatwia pakowanie modeli uczenia maszynowego do użytku przez różne narzędzia downstream, takie jak Apache Spark.

Platforma jest również dostępna w wersji komercyjnej o nazwie Managed MLflow [7]. Rozszerza funkcjonalność, skupiając się na niezawodności, bezpieczeństwie i skalowalności w kontekście przedsiębiorstw. Demonstruje to, jak platforma open-source może być podstawą dla bardziej zaawansowanego, komercyjnego produktu.

Podsumowując, MLflow cieszy się powodzeniem, ponieważ jest łatwy w użyciu lokalnie bez dedykowanego serwera i ma przyjazny interfejs użytkownika, w którym można eksplorować swoje eksperymenty.

### Metaflow[8]

Netflix stworzył Metaflow jako platformę MLOps open-source do budowania i zarządzania projektami nauki o danych na dużą skalę, na poziomie przedsiębiorstwa. Naukowcy zajmujący się danymi mogą korzystać z tej platformy do end-to-end rozwoju i wdrażania swoich modeli uczenia maszynowego.

- **Obsługa bogatej biblioteki.** Metaflow jest zintegrowany z szeroką gamą popularnych narzędzi nauki o danych, w tym TensorFlow i scikit-learn, umożliwiając użytkownikom kontynuację pracy z preferowanymi narzędziami. Wsparcie dla języków programowania Python i R zapewnia elastyczność w wyborze bibliotek i pakietów.

- **Zaawansowane zarządzanie wersjami.** Jedną z kluczowych cech Metaflow jest automatyzacja wersjonowania i śledzenia eksperymentów związanych z uczeniem maszynowym. Użytkownicy mogą łatwo śledzić i porównywać wyniki różnych eksperymentów, co ułatwia zarządzanie procesem rozwoju modelu.

Metaflow zostało wyspecjalizowane do tworzenia uczenia maszynowego na dużą skalę. Dlatego jest to idealne rozwiązanie dla dużych zespołów analityków danych, którzy chcą zbudować i wdrożyć modele uczenia maszynowego w dużych przedsiębiorstwach.

### Flyte[11]

Flyte to orkiestrator typu open-source, który został zaprojektowany z myślą o uproszczeniu procesu tworzenia stabilnych przepływów pracy, integrujących zarządzanie danymi i uczenie maszynowe, przeznaczonych do etapu produkcyjnego. Architektura Flyte kładzie szczególny nacisk na skalowalność i powtarzalność, wykorzystując Kubernetes jako podstawową platformę.

Flyte oferuje mnóstwo funkcji i przypadków użycia, od prostego projektu uczenia maszynowego po złożone projekty LLM. Główne cechy Flyte to:

- **Wsparcie dla projektów na dużą skalę.** Flyte pomaga im realizować obliczenia na dużą skalę, które są kluczowe dla ich działalności.

- **Wysoka powtarzalność.** To narzędzie może również pomóc w zapewnieniu powtarzalności modeli uczenia maszynowego,

- **Wsparcie wielojęzyczne.** Flyte zostało stworzone do obsługi złożonych projektów ML w Pythonie, Javie lub Scala.

Z materiałów marketingowych[15] dowiadujemy się, że Flyte było testowane wewnętrznie przez Lyft przed jego publicznym wydaniem. Narzędzie to ma udowodniony rekord zarządzania ponad 7 000 unikalnymi przepływami pracy, które łącznie osiągają 100 000 wykonanych operacji każdego miesiąca.

### MLReef[12]

MLReef to platforma MLOps przeznaczona do współpracy zespołów i udostępniania wyników eksperymentów z uczenia maszynowego. Projekty są budowane na modułach, które mogą być tworzone przez użytkownika lub społeczność, co przyspiesza rozwój i czyni przepływ pracy bardziej efektywnym poprzez promowanie współbieżności.

MLReef dostarcza narzędzia w czterech kierunkach:

- **Zarządzanie danymi.** Pełna wersja hostingu danych i infrastruktury przetwarzania dla ustawienia i zarządzania modelami uczenia maszynowego.
- **Repozytoria skryptów.** Każdy programista ma dostęp do zkonteneryzowanych i wersjonowanych repozytoriów skryptów, które można wykorzystać w potokach uczenia maszynowego.
- **Zarządzanie eksperymentami.** MLReef umożliwia śledzenie eksperymentów w różnych iteracjach projektu.
- **MLOps.** Rozwiązanie to pomaga optymalizować zarządzanie potokami i orkiestrację, automatyzując rutynowe zadania.

MLReef jest przyjazny dla projektów o każdej wielkości:

- **Dla początkujących.** Użytkownik znajdzie przyjazny interfejs i wsparcie społeczności.
- **Dla doświadczonych.** MLReef umożliwia budowanie projektu na Git, jednocześnie zajmując się wszystkimi problemami związanymi z DevOps.
- **Dla przedsiębiorstw.** MLReef jest łatwy do skalowania i kontroli w chmurze lub lokalnie.

### Seldon Core[13]

Seldon Core to platforma do wdrażania modeli uczenia maszynowego na Kubernetes. Pomaga programistom budować modele w solidnym środowisku Kubernetes, oferując funkcje takie jak niestandardowe definicje zasobów do zarządzania grafami modeli. Można również integrować narzędzia ciągłej integracji i wdrażania z platformą.

Główne cechy Seldon Core:

- **Budowanie skalowalnych modeli.** Konwersja modeli z TensorFlow, PyTorch, H2O i innych na skalowalną architekturę mikroserwisów opartą na REST/GRPC.
- **Monitorowanie wydajności modeli.** Automatyczne skalowanie i zaawansowane rozwiązania do mierzenia wydajności modeli, wykrywania anomalii i prowadzenia testów A/B.
- **Solidność i niezawodność.** System wspierany przez ciągłe utrzymanie i aktualizacje polityki bezpieczeństwa.

### Sematic[14]

Sematic to otwarta platforma rozwoju uczenia maszynowego, umożliwiająca inżynierom ML i naukowcom danych tworzenie złożonych, potoków end-to-end uczenia maszynowego za pomocą prostego kodu Python, który może być wykonywany na różnych platformach: lokalnie, na wirtualnej maszynie, w chmurze lub na klastrze Kubernetes.

Cechy oferowane przez Sematic:

- **Łatwe rozpoczęcie pracy.** Bez początkowego wdrażania lub wymagań infrastrukturalnych.
- **Kompatybilność środowiska lokalnego i chmurowego.** Ten sam kod uruchomiony na laptopie można bezproblemowo wykonać na klastrze Kubernetes.
- **Przejrzystość end-to-end.** Każdy artefakt potoku jest starannie przechowywany, śledzony i prezentowany w internetowym pulpicie nawigacyjnym.
- **Wykorzystanie zróżnicowanych zasobów obliczeniowych.** Optymalizacja wydajności i śladu chmurowego poprzez różne opcje, w tym CPU, pamięć, GPU i klastry Spark.
- **Skupienie się na reproduktywności.** Ponowne uruchamianie potoków z intuicyjnego interfejsu użytkownika.

## II.3. Uzasadnienie wyboru orkiestratora

W ramach pracy dyplomowej za platformę wybrany został Kubeflow, ponieważ jest to najbardziej popularna otwartoźródłowa platforma MLOps dostępna na rynku. Jest to również jedna z najstarszych, gdyż została stworzona przez Google w 2017 roku. Autorzy [17] podkreślają, że Kubeflow został stworzony jako sposób na uruchamianie przepływów pracy związanych z uczeniem maszynowym na Kubernetes. Platforma ta jest wykorzystywana między innymi do trenowania i serwowania modeli uczenia maszynowego w różnych środowiskach (np. lokalnym i w chmurze), korzystania z Jupyter Notebooks do zarządzania zadaniami związanymi z trenowaniem modeli (nie tylko TensorFlow), uruchamiania zadań trenujących wykorzystujących zasoby (takie jak dodatkowe CPU lub GPU), które nie są dostępne na osobistym komputerze, oraz łączenia kodu uczenia maszynowego z różnych bibliotek.

Jest również świetną opcją dla infrastruktury w dużych przedsiębiorstwach, ponieważ umożliwia łatwą integrację systemu modelowania opartego na notatnikach z procesami ETL (Extract, Transform, Load, czyli Ekstrakcja, Transformacja, Ładowanie) na lokalnym data lake lub w chmurze. Procesy ETL polegają na ekstrakcji danych z różnych źródeł, ich transformacji do odpowiedniego formatu i struktury, oraz ładowaniu do docelowych systemów do dalszej analizy. Obsługuje także wielodostępność na różnych sprzętach, zarządzając aspektem orkiestracji kontenerów infrastruktury.

Autorzy [18] z kolei podkreślają, że Kubeflow rozwiązuje szereg problemów związanych z wdrażaniem przepływów pracy uczenia maszynowego na Kubernetes. Bezpośrednie korzystanie z API Kubernetes jest zbyt niskopoziomowe dla większości naukowców zajmujących się danymi. Taka osoba musi znać wiele technik i technologii

bez konieczności dodawania złożoności API Kubernetes do listy. Kubeflow zapewnia szybsze i bardziej spójne wdrożenie, lepszą kontrolę nad portami i dostępem do komponentów dla zwiększenia bezpieczeństwa, ochronę przed nadmiernym alokowaniem zasobów, co pozwala na oszczędności kosztów.

Na oficjalnym blogu[23] w październiku 2022 r. Google ogłosił, że projekt Kubeflow złożył wniosek o dołączenie do Cloud Native Computing Foundation(CNCF). W lipcu 2023 r. fundacja zagłosowała[24] za przyjęciem jako projektu na etapie inkubacji. Jak widać Kubeflow jest projektem, który stale się rozwija i jest aktywnie wspierany przez społeczność. Warto zauważyć, że jest jednym z najbardziej aktywnych projektów w ramach CNCF zajmując 22 miejsce w rankingu[25], ponad 13 000 osób dodało ten projekt do swojej prywatnej listy ulubionych projektów na GitHub[16]. Projekt ten posiada aż 199 autorów(w porównaniu do 3 na początku) oraz 22 wpierające organizacje.

Podsumowując, Kubeflow to najlepszy wybór, ponieważ jest to wszechstronna, dobrze wspierana i niezwykle funkcjonalna platforma MLOps open-source, która skutecznie integruje się z Kubernetes, zapewniając rozwiązania dla szeregu wyzwań związanych z uczeniem maszynowym i operacjami na dużą skalę.

# Rozdział III. Kubeflow

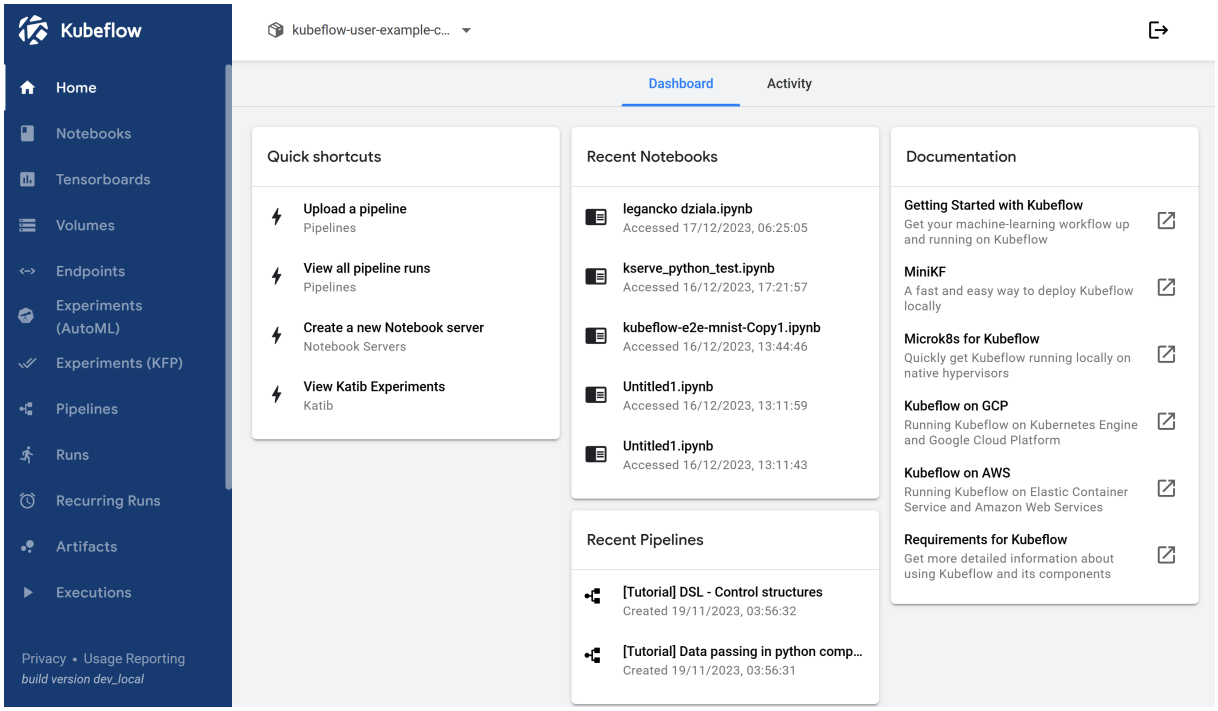
## III.1. Komponenty Kubeflow

Autorzy [18] opisują Kubeflow jako platformę, która składa się z kilku komponentów, które są zintegrowane z Kubernetes. Jest to platforma modularna w której można dodawać lub odejmować komponenty. Pozwala to również na tworzenie własnych komponentów. Istnieje wiele komponentów tworzonych przez społeczność lub firmy zewnętrzne.

W skład postawowych głównych komponentów wchodzi:

1. Dashboard
2. Notebooks
3. Pipelines
4. AutoML(Katib)
5. Model Training
6. Model Serving(Kserve)

### III.1.1. Dashboard




**Rys. 2. Panel Nawigacyjny(Dashboard) Kubeflow**

Źródło: Badanie własne.

Pulpit nawigacyjny (Dashboard) jest to miejsce, które scala wszystkie komponenty w jedno miejsce. Dzięki niemu nie musimy logować się do każdego serwisu oddzielnie.

W podstawowej konfiguracji składa się z następujących elementów:

- **Home.** Strona główna platformy Kubeflow, z której można przejść do różnych sekcji dashboardu.

- **Notebooks.** Sekcja umożliwiająca użytkownikowi zarządzanie środowiskami Jupyter, Visual Studio Code i RStudio, które są interaktywnymi narzędziami do kodowania i analizy danych.

- **Volumes.** Tutaj użytkownicy mogą zarządzać woluminami danych, które są wykorzystywane do przechowywania i dostępu do danych przez aplikacje w Kubeflow.

- **Endpoints.** Sekcja umożliwiająca tworzenie i zarządzanie punktami końcowymi, które służą do wdrażania modeli ML jako usług (model serving).

- **Experiments (AutoML).** Miejsce, gdzie można zarządzać eksperymentami z automatycznego uczenia maszynowego (AutoML).

- **Pipelines.** Panel do tworzenia i zarządzania potokami (pipelines), które automatyzują przepływ pracy uczenia maszynowego.

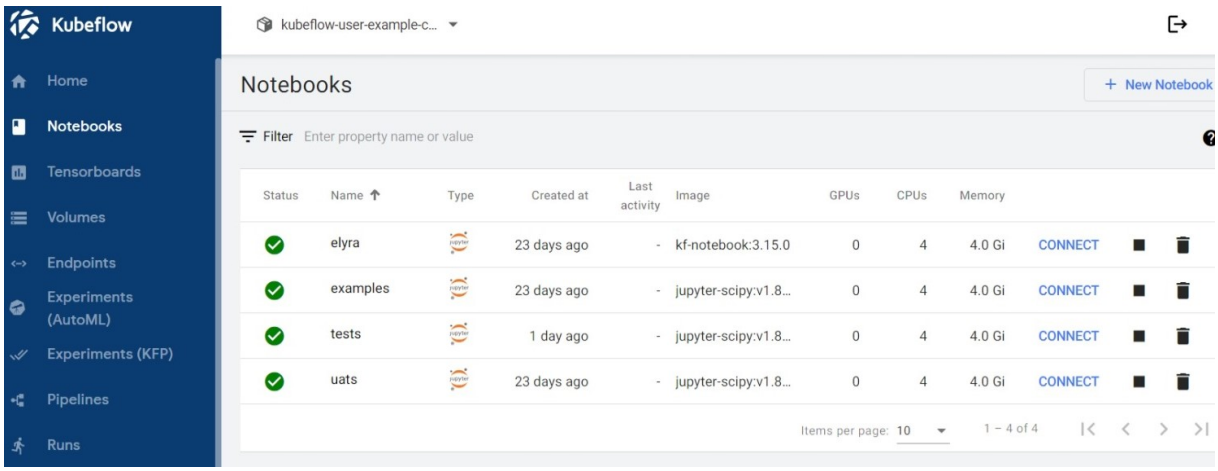
- **Runs.** Tutaj wyświetlane są uruchomienia potoków, które można analizować i monitorować.

- **Recurring Runs.** Sekcja dla cyklicznie powtarzających się uruchomień potoków.

- **Artifacts.** Elementy, które są wytworzone podczas uruchomienia potoków, np. modele, dane wyjściowe.

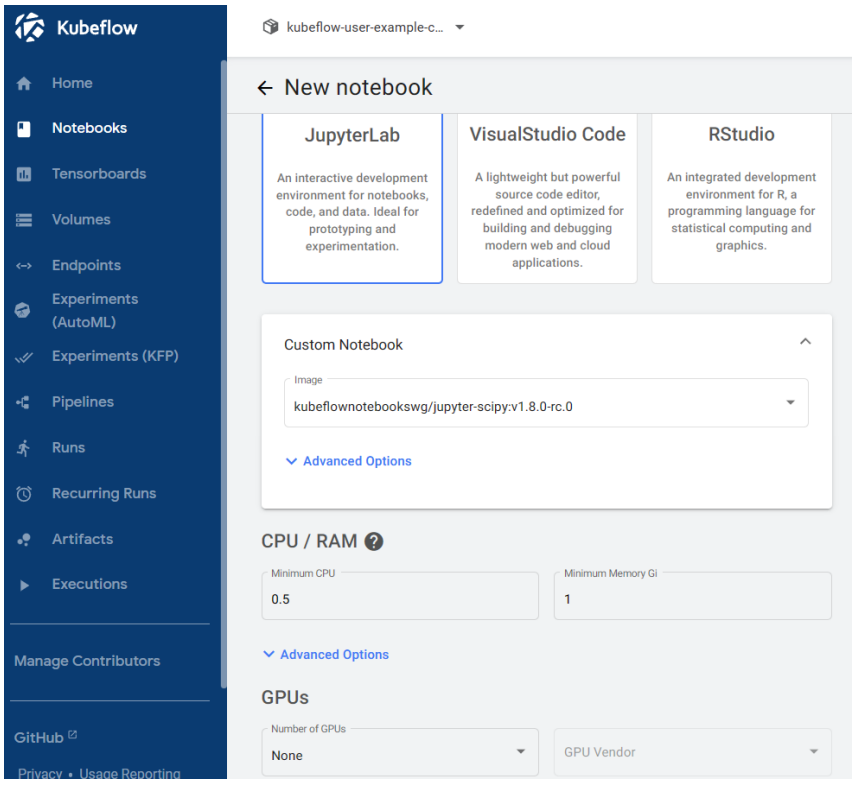
- **Executions.** Zarządzanie wykonaniami, czyli konkretne instancje uruchomionych potoków.

### III.1.2. Kubeflow Notebooks




**Rys. 3. Panel Kubeflow Notebooks**

Źródło: Badanie własne.

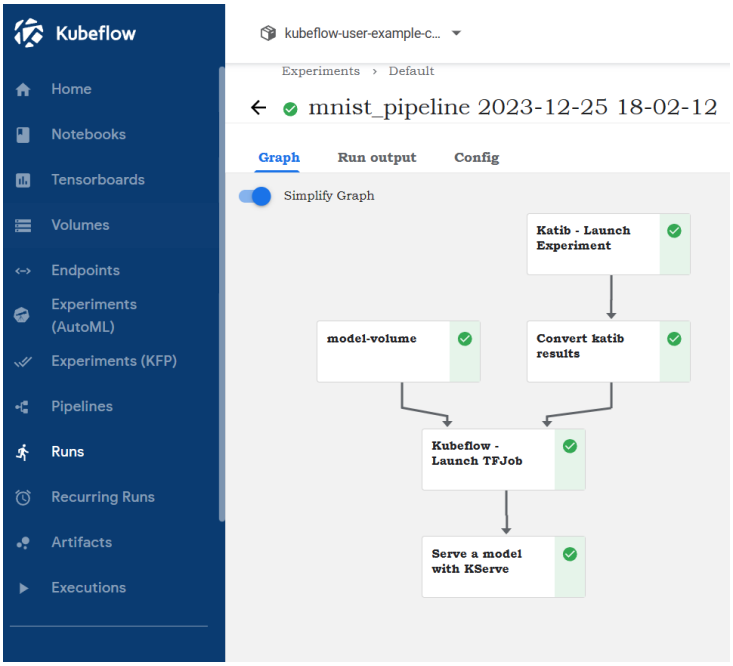



**Rys. 4. Panel tworzenia Kubeflow Notebooks**

Źródło: Badanie własne.

Kubeflow Notebooks pozwala na tworzenie środowisk Jupyter Notebook, Visual Studio Code i RStudio w ramach jednej platformy. Tworzenie odbywa się z poziomu interfejsu widocznego na rysunku 3 z opcją dostosowywania obrazów bazowych oraz zasobów sprzętowych pokazanym na rysunku 4 według wymogów obliczeniowych danego projektu. Proces współdzielenia notatników między użytkownikami jest uproszczony, co sprzyja kolaboracji. Dzięki wolumenom, zapewnia możliwość przechowywania danych w sposób trwały, co pozwala na zachowanie stanu pracy między sesjami. Notatniki mogą być także używane do budowy komponentów w ramach Kubeflow Pipelines, co jest elementem procesu automatyzacji uczenia maszynowego. Pozwala to na transformację prototypów w gotowe przepływy pracy, które mogą być następnie używane w dalszym procesie wdrażania modeli.

### III.1.3. Pipelines



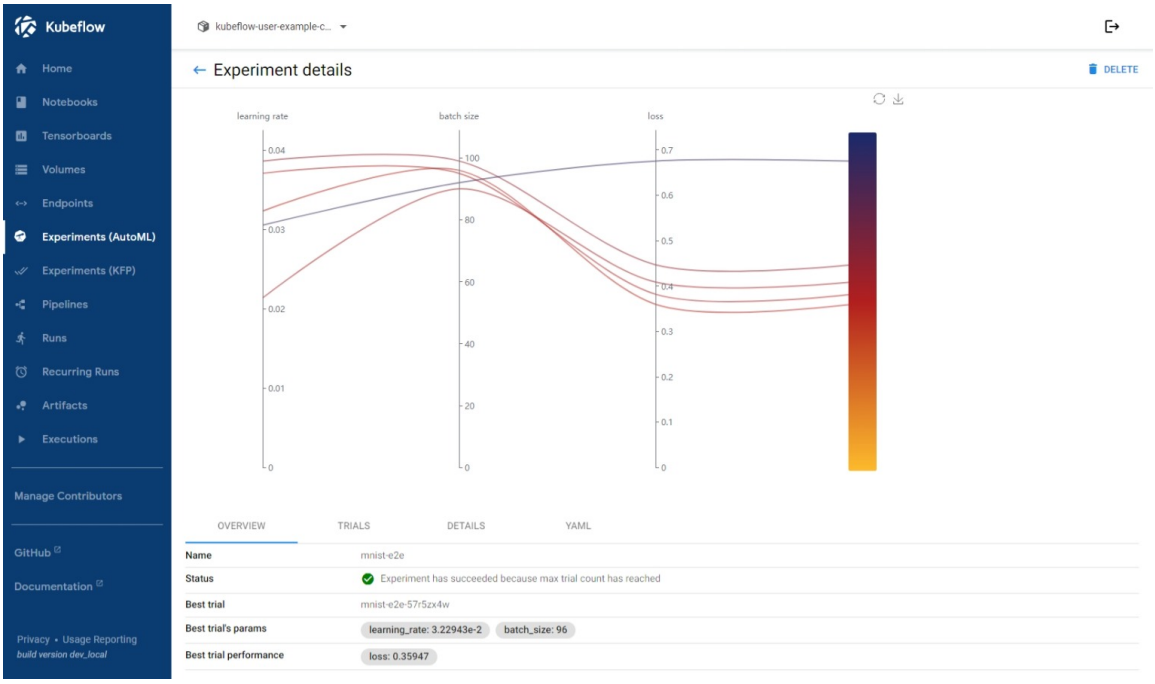

**Rys. 5. Przykładowy przepływ pracy w Kubeflow Pipelines**

Źródło: Badanie własne.

Kubeflow Pipelines to komponent platformy Kubeflow, który umożliwia definiowanie, wdrażanie i zarządzanie potokami uczenia maszynowego. Jest to zestaw narzędzi przeznaczony do orkiestracji zadań związanych z uczeniem maszynowym, pozwalający na budowę kompleksowych przepływów pracy, które są zarówno skalowalne, jak i powtarzalne. Użytkownicy mogą projektować potoki za pomocą Python SDK, a następnie wdrażać je w środowisku Kubernetesa, gdzie każdy etap potoku reprezentowany jest przez kontener.

Kubeflow Pipelines umożliwiają automatyzację procesów takich jak przetwarzanie danych, trening modeli, ich walidację i wdrażanie, co przyczynia się do efektywniejszego i bardziej uporządkowanego procesu wytwarzania oprogramowania w dziedzinie uczenia maszynowego. Efekt uruchomionego potoku zaprezentowane jest na rysunku 5.

### III.1.4. Katib




**Rys. 6. Przykład eksperymentu w Katib**

Źródło: Badanie własne.

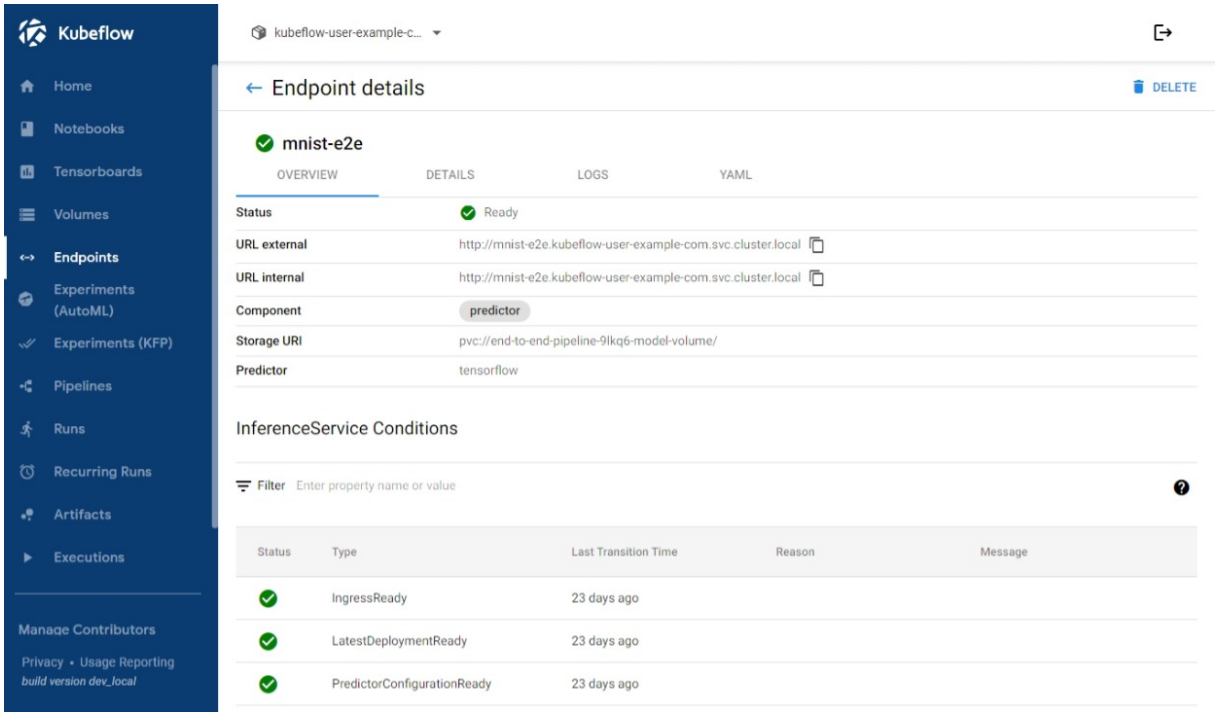
Katib to komponent dedykowany automatyzacji procesu doboru hiperparametrów, czyli zewnętrznych konfiguracji algorytmów uczenia maszynowego, które nie są bezpośrednio uczące się z danych. Dobór hiperparametrów jest istotny, ponieważ wpływa na wydajność i skuteczność modelu uczenia maszynowego. Katib umożliwia przeprowadzanie szeroko zakrojonych eksperymentów w celu ustalenia optymalnych wartości hiperparametrów, wykorzystując różne strategie, takie jak przeszukiwanie siatkowe, losowe czy optymalizacja bayesowska.

Działa w oparciu o możliwości Kubernetes, co pozwala na efektywne skalowanie i zarządzanie procesami eksperymentalnymi. Użytkownik określa zakres i przestrzeń poszukiwań hiperparametrów, a Katib automatycznie przeprowadza eksperymenty, analizuje wyniki i wybiera najlepsze rozwiązania, co przyspiesza i usprawnia proces budowania modeli. Przykładowe działanie przedstawione jest na rysunku 6

### III.1.5. Kubeflow Training Operator

Jest to zestaw narzędzi umożliwiającym użytkownikom efektywne zarządzanie, skalowanie i wdrażanie procesów treningowych, wykorzystując potencjał klastra Kubernetes. Training Operator obsługuje różne ramy pracy uczenia maszynowego, takie jak TensorFlow, PyTorch, MXNet i inne, dostarczając spójny interfejs do kontrolowania i monitorowania treningu modeli. Dzięki temu, proces treningu może być bardziej automatyczny, a zasoby są alokowane oraz zarządzane w sposób elastyczny i optymalny. Narzędzie to jest kluczowe dla operacyjnej efektywności w szkoleniu modeli, umożliwiając szybsze iteracje i lepsze wykorzystanie infrastruktury.

### III.1.6. KServe




**Rys. 7. Przykład wdrożenia modelu w Kubeflow**

Źródło: Badanie własne.

Kserve to komponent Kubeflow przeznaczony do serwingu modeli uczenia maszynowego, ułatwiający ich wdrażanie jako usługi webowe. Zintegrowany z Kubernetes, Kserve wykorzystuje jego możliwości do zarządzania kontenerami, co gwarantuje wysoką dostępność i skalowalność serwowanych modeli. Przykład rakiego rozwiązania zaprezentowany jest na rysunku 7.

Główne funkcjonalności Kserve obejmują:

- **Automatyczne skalowanie.** Dostosowuje ilość zasobów w zależności od obciążenia usług serwingu modeli.

- **Zarządzanie wersjami.** Umożliwia łatwe zarządzanie różnymi wersjami wdrożonych modeli.

- **Abstrakcja infrastruktury.** Minimalizuje złożoność zarządzania infrastrukturą serwingu modeli.

Ponadto, Kserve oferuje zaawansowane techniki zarządzania wersjami i testowania modeli, takie jak:

- **A/B testing.** Testowanie dwóch wersji modelu, aby określić, która lepiej spełnia określone kryteria wydajności.

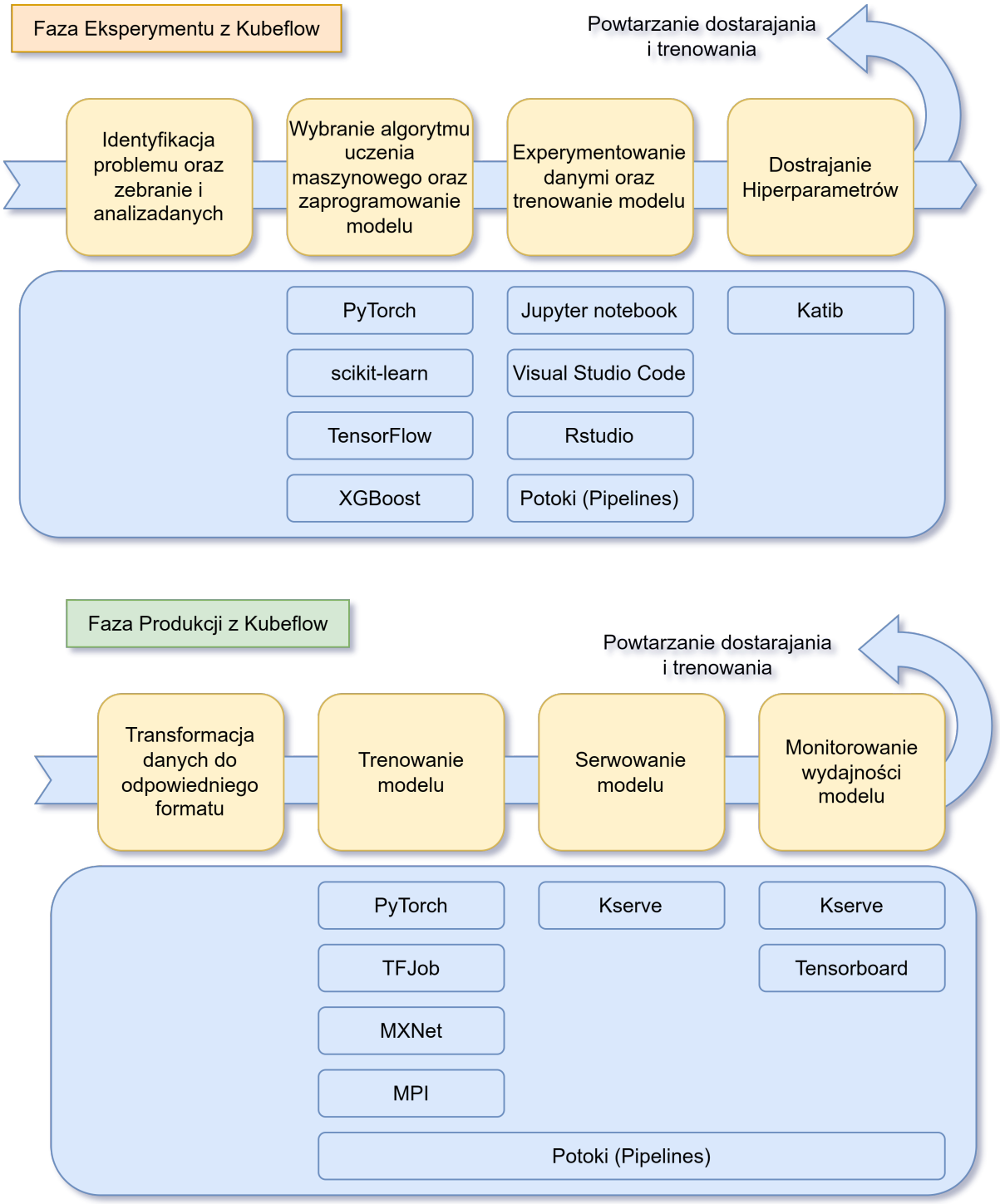
- **Multi-armed bandit.** Algorytm do optymalizacji wyboru wersji modelu w czasie rzeczywistym, bazujący na wstępnych wynikach.

- **Progresywne wdrażanie (canary rollouts).** Stopniowe wprowadzanie nowej wersji modelu, co pozwala na monitorowanie wydajności i stabilności przed pełnym wdrożeniem.

Te funkcje wspierają kompleksowe zarządzanie cyklem życia modelu, od jego wdrożenia, przez monitorowanie, aż po aktualizacje, w zmiennych warunkach produkcyjnych.

### III.1.7. Wnioski

Podsumowując wszystkie te narzędzia razem tworzą przedstawione na rysuku 8 środowisko pozwalające stworzyć, uczyć i dostarczyć modele. Do każdego kroku zaznaczonego na żółto istnieje wiele możliwych narzędzi do wykorzystania zaznaczonych na niebiesko. Platforma ta wspiera dużo popularnych technologii, bibliotek oraz algorytmów. Gdyby nie było zintegrowanego środowiska takiego jak Kubeflow przy każdej zmianie technologii konieczna byłaby zmiana części lub całości narzędzi.

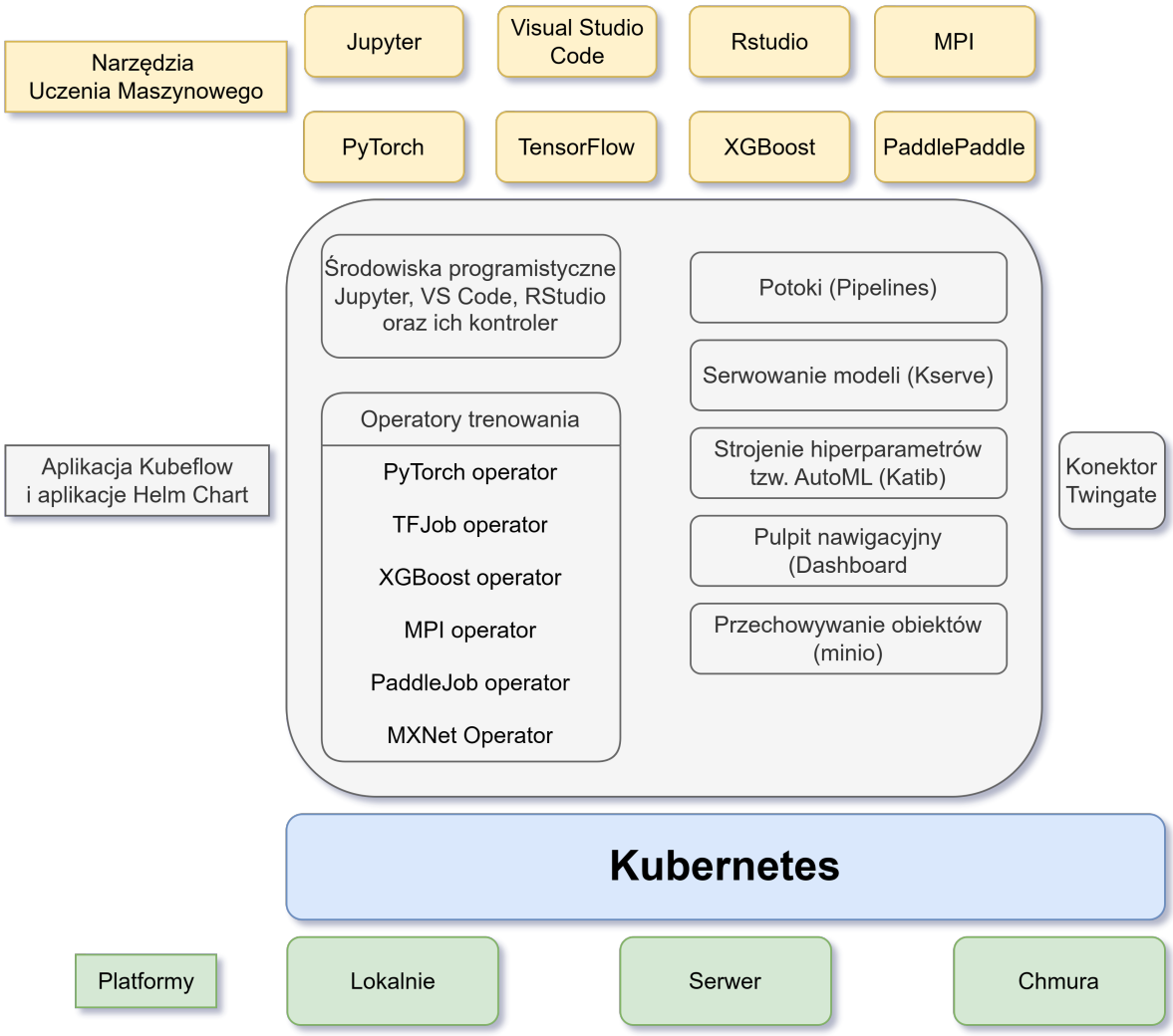



**Rys. 8. Przepływ pracy w Kubeflow**

Źródło: Na podstawie https://www.kubeflow.org/docs/started/architecture/

# Rozdział IV. Implementacja

## IV.1. Wprowadzenie




**Rys. 9. Architentura implementacji**

Źródło: Na podstawie https://www.kubeflow.org/docs/started/architecture/

Oficjalna dokumentacja Kubeflow[5] wymienia 2 główne sposoby instalacji Kubeflow:

1. Używanie dystrybucji pakietowych
2. Używanie surowych manifestów (dla zaawansowanych użytkowników)

Do implementacji rozwiązania przedstawionego na 9 zostały użyte surowe manifesty, czyli plliki konfiguracyjne Kubernetes. Do dostosowania ich do potrzeb pracy

dyplomowej użyte zostało narzędzie Kustomize. Dostępem do środowiska zarządza rozwiązanie typu zero trust Twingate. Natomiast do uruchamiania procesów CI/CD wykorzystywany jest Git i GitHub Actions.

### IV.1.1. Dystrybucje Kubeflow

Dystrybucje Kubeflow to spersonalizowane wersje platformy Kubeflow, które są dostosowane do konkretnych potrzeb i środowisk. Każda dystrybucja zawiera zestaw składników Kubeflow, które zostały zintegrowane i skonfigurowane, aby działać sprawnie na określonej platformie lub w określonym środowisku. Przykłady takich dystrybucji obejmują Kubeflow na AWS, Azure, Google Cloud, IBM Cloud i inne.

#### Zalety Dystrybucji Kubeflow:

1. **Dostosowanie do Platformy.** Każda dystrybucja jest dostosowana do konkretnej platformy chmurowej lub środowiska, co ułatwia wdrażanie i zarządzanie.

2. **Zoptymalizowana Integracja.** Dostarczają zoptymalizowanej integracji z usługami i funkcjami specyficznymi dla danej platformy, np. systemami przechowywania danych.

3. **Wsparcie od Dostawców.** Często oferują wsparcie i aktualizacje od firm, które utrzymują te dystrybucje, co może być korzystne dla przedsiębiorstw.

4. **Łatwość Użycia.** Uproszczony proces instalacji i konfiguracji, co jest przydatne dla użytkowników, którzy nie są głęboko zaznajomieni z Kubernetes lub Kubeflow.

#### Wady Dystrybucji Kubeflow:

1. **Ograniczenia Specyficzne dla Platformy.** Mogą być ograniczone do funkcji i możliwości oferowanych przez określoną platformę, co może nie być odpowiednie dla wszystkich scenariuszy użycia.

2. **Zależność od Dostawcy.** Zwiększona zależność od konkretnego dostawcy może prowadzić do tzw. "vendor lock-in", ograniczając elastyczność w zmianie platform.

3. **Różnice w Funkcjonalnościach.** Różne dystrybucje mogą oferować różne zestawy funkcji, co może powodować niejednolitość doświadczeń dla użytkowników korzystających z różnych dystrybucji.

4. **Potencjalne Opóźnienia w Aktualizacjach.** Możliwe opóźnienia w implementacji najnowszych funkcji Kubeflow, ponieważ aktualizacje muszą być najpierw zintegrowane i przetestowane przez dostawców dystrybucji.

### IV.1.2. Manifesty Kubeflow

Surowe manifesty Kubeflow to podstawowe, niezmodyfikowane pliki konfiguracyjne używane do instalacji i konfiguracji Kubeflow w środowisku Kubernetes. Są one przeznaczone głównie dla zaawansowanych użytkowników, którzy chcą mieć pełną kontrolę nad procesem instalacji i konfiguracji platformy.

#### Zalety Manifestów Kubeflow:

1. **Pełna Kontrola.** Użytkownik ma pełną kontrolę nad konfiguracją i może dostosować instalację do specyficznych potrzeb i wymagań.

2. **Elastyczność.** Pozwala na bardziej elastyczne wdrożenie, ponieważ użytkownicy mogą wybrać tylko te komponenty, które są im potrzebne.

3. **Zrozumienie Procesu.** Instalacja za pomocą surowych manifestów wymaga głębszej wiedzy na temat Kubeflow i Kubernetes, co może przyczynić się do lepszego zrozumienia działania platformy.

4. **Aktualizacje i Modyfikacje.** Ułatwia wdrażanie niestandardowych aktualizacji i modyfikacji, które mogą nie być dostępne w pakietowanych dystrybucjach.

#### Wady Manifestów Kubeflow:

1. **Złożoność.** Proces instalacji i konfiguracji jest znacznie bardziej skomplikowany i czasochłonny niż w przypadku użycia pakietowanych dystrybucji.

2. **Brak Wsparcia.** Brak wsparcia ze strony społeczności Kubeflow dla problemów specyficznych dla środowiska, co może być wyzwaniem, zwłaszcza dla mniej doświadczonych użytkowników.

3. **Ryzyko Błędów.** Zwiększone ryzyko popełnienia błędów podczas konfiguracji, co może prowadzić do niestabilności lub problemów z bezpieczeństwem.

4. **Konserwacja i Aktualizacje.** Utrzymanie i aktualizowanie instalacji wymaga ciągłego monitorowania i interwencji, co może być obciążeniem dla zespołów IT.

Do instalacji Kubeflow w ramach tej pracy dyplomowej wykorzystane zostały surowe manifesty, aby mieć pełną kontrolę nad procesem instalacji i konfiguracji platformy. Pomaga to również zrozumieć proces instalacji oraz jej komponentów, aby lepiej zaznajomić się z platformą.

### IV.1.3. Kubernetes

Kubernetes jest otwartoźródłowym systemem służącym do automatyzacji wdrażania, skalowania i zarządzania aplikacjami skonteneryzowanymi. Został zaprojektowany przez Google i jest rozwijany przez Cloud Native Computing Foundation. Kubernetes jest

platformą, która umożliwia zarządzanie zasobami obliczeniowymi, takimi jak obliczenia, sieć i pamięć, w sposób zdecentralizowany. Jest to możliwe dzięki zastosowaniu orkiestracji kontenerów, która pozwala na automatyzację procesów związanych z wdrażaniem, skalowaniem i zarządzaniem aplikacjami skonteneryzowanymi.

Do instalacji wykorzystany został microk8s od Canonical. Jest to lokalna instalacja Kubernetes, która jest szybka i lekka. Microk8s jest również certyfikowanym dostawcą Kubernetes, co oznacza, że jest zgodny z Kubernetes i spełnia wymagania CNCF. Producent na stronie głównej[26] podaje, że spełnia wszystkie założenia, żeby być platformą produkcyjną, gdyż posiada obsługę wielu węzłów, wysoką dostępność, automatyczne aktualizacje, oraz wsparcie dla prywatnych klastrów. Zaletą jest także ścisła integracja z Ubuntu, microk8s jest dostępny jako snap, co oznacza, że jest zainstalowany i gotowy do użycia w kilka sekund. Korzysta z containerd lub kata containers, co jest dużym plusem.. Pozwala to na wykorzystywanie wszystkich zasobów komputera, a nie tylko tych, które są dostępne dla dockera. Oznacza to łatwą integrację z GPU lub FPGA i innymi zasobami sprzętowymi. Dodatkową zaletą microk8s jest obsługa dodatków, które bardzo nam ułatwią instalację Kubeflow. Są to gotowe rozwiązania, które można zainstalować na klastrze Kubernetes

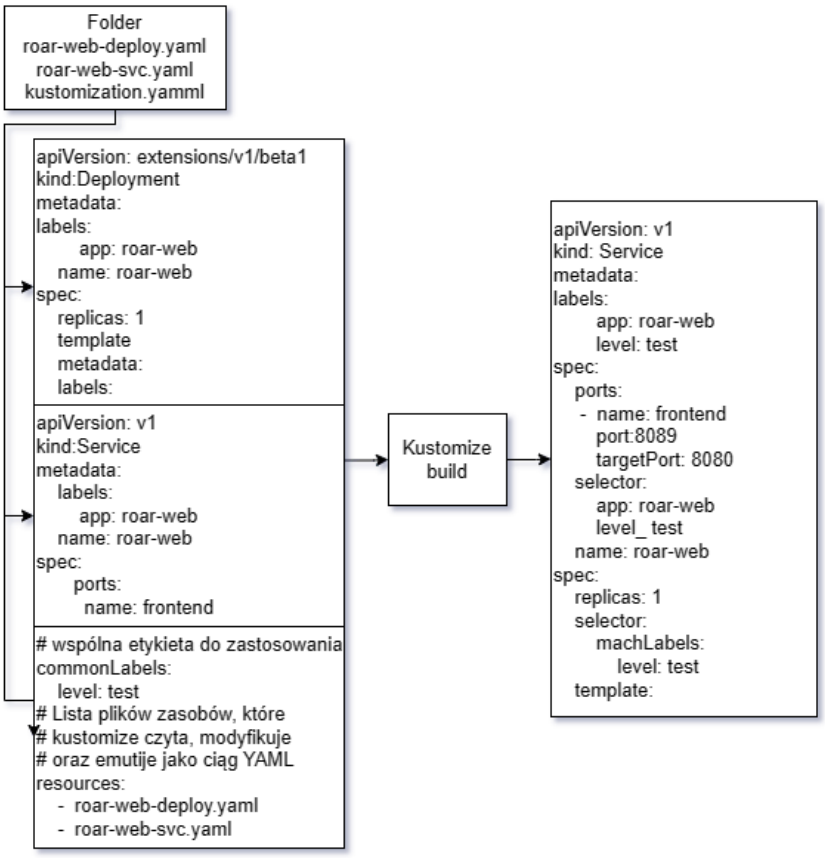
- **cert-manager** - automatyczne zarządzanie certyfikatami TLS w klastrze Kubernetes
- **dns** - obsługa DNS dla klastra Kubernetes
- **helm** - zarządzanie pakietami Helm Chart dla klastra Kubernetes
- **hostpath-storage** - obsługa przechowywania danych dla klastra Kubernetes
- **metallb** - obsługa Load Balancer dla klastra Kubernetes
- **ingress** - obsługa Ingress dla klastra Kubernetes
- **metrics-server** - zbieranie metryk dla klastra Kubernetes
- **prometheus** - obsługa prometheus i grafana dla klastra Kubernetes
- **registry** - obsługa prywatnego rejestru obrazów typu Docker dla klastra Kubernetes
- **gpu** - obsługa GPU dla klastra Kubernetes
- **minio** - obsługa systemu przechowywania plików dla klastra Kubernetes
- **Oraz wiele innych dodatków przygotowanych przez społeczność**

### IV.1.4. Kustomize

Kustomize, opisane przez Autorów [27], to narzędzie do generowania niestandardowych wersji manifestów Kubernetes poprzez nakładanie deklaratywnych specyfikacji na istniejące. Działa przez zachowanie oryginalnych plików manifestu, umożliwiając równoczesne utworzenie ich zmodyfikowanych wersji. Wykorzystuje plik kustomization.yaml do definiowania zmian, które mają być zastosowane. Kustomize umożliwia łatwe zarządzanie wieloma wariantami konfiguracji, stosowanie łat (patches) i generowanie zasobów, zachowując przy tym czytelność i zarządzalność kodu. Jest zintegrowane z kubectl, upraszczając proces wdrażania i zarządzania manifestami w Kubernetes.

Autorzy pokazali to na przykładzie aplikacji webowej wraz z bazą MySQL. W folderze z plikami konfiguracyjnymi znajdują się następujące pliki:

- roar-web-deploy.yaml - Plik manifestu Kubernetes dla aplikacji webowej
- roar-web-svc.yaml - Plik manifestu Kubernetes dla usługi aplikacji webowej
- kustomization.yaml - Plik wejściowy Kustomize deklarujący rodzaje przekształceń, które mają zostać wprowadzone do manifestów.




**Rys. 10. Przykład użycia Kustomize**

Źródło: Na podstawie przykładu z raportu "Leveraging Kustomize for Kubernetes Manifests"[27]

Jak widać na rysunku 10 po lewej widzimy pliki roar-web-deploy.yaml, roar-web-svc.yaml oraz kustomization.yaml. Wywołanie komendy kustomize build w folderze gdzie znajdują się poprzednio wymienione pliki zwraca nam gotową do wdrożenia konfigurację kubernetesową złożoną z plików roar-web-deploy.yaml oraz roar-web-svc.yaml wraz z naniesionymi konfiguracjami z pliku kustomization.yaml. Widać to wyraźnie patrząc na etykiety level:test. Dzięki temu narzędziu możemy łatwo zarządzać wieloma wariantami konfiguracji, stosować łaty (patches) i generować zasoby, zachowując przy tym czytelność i zarządzalność kodu.

### IV.1.5. Twingate

Twingate[30] to platforma umożliwiająca zarządzanie i automatyzację bezpiecznego dostępu do infrastruktury technicznej, zarówno lokalnie, jak i w chmurze. Działa jako centralna warstwa orkiestracji w modelu Zero Trust, który jest strategią cyberbezpieczeństwa zakładającą, że żadne urządzenie ani użytkownik nie powinny być zaufane domyślnie, niezależnie od ich lokalizacji czy wcześniejszych działań w sieci. W podejściu Zero Trust, każda próba dostępu jest weryfikowana, uwierzytelniana i autoryzowana w ścisłej zgodności z zasadami bezpieczeństwa, co zapewnia tworzenie ekosystemu bezpieczeństwa najwyższej klasy bez konieczności rekonstrukcji istniejącej sieci. Kluczową cechą Twingate jest to, że żaden pojedynczy komponent nie może samodzielnie podjąć decyzji o przepuszczaniu ruchu do innego komponentu lub zasobu w sieci zdalnej. Platforma oferuje funkcje takie jak zastępowanie VPN, zdalny dostęp do prywatnych zasobów i sieci, zarządzanie dostępem do infrastruktury technicznej, kontrola urządzeń, dostęp oparty na adresie IP, bezpieczeństwo internetowe oraz spełnianie wymagań związanych z zgodnością.

### IV.1.6. Git

Autorzy "Pro Git"[29] opisują Git jako rozproszony systemem kontroli wersji, zaprojektowany do efektywnego zarządzania projektami o różnej wielkości, od małych osobistych inicjatyw po bardzo duże przedsięwzięcia. Został stworzony przez Linusa Torvaldsa w 2005 roku, głównie z myślą o ułatwieniu pracy nad jądrem systemu Linux. Git pozwala deweloperom na śledzenie zmian w kodzie, współpracę z innymi oraz efektywne zarządzanie historią projektu.

**Podstawowe funkcje Git:**

1. **Commit.** Jest to podstawowa operacja w Git, pozwalająca na zapisanie stanu zmian w repozytorium. Commit reprezentuje kompletny zestaw zmian w kodzie od ostatniego commitu i jest identyfikowany za pomocą unikalnego identyfikatora SHA-1. Każdy commit zawiera informacje o autorze, wiadomość opisującą zmiany oraz wskaźnik do poprzednich commitów, umożliwiając śledzenie historii zmian.

2. **Push.** Komenda push służy do przesyłania lokalnych zmian (commitów) do zdalnego repozytorium. Dzięki tej operacji inni użytkownicy pracujący nad projektem mogą zobaczyć dokonane zmiany. Push aktualizuje zdalne repozytorium, ale nie

wpływa na stan innych lokalnych repozytoriów, aż do momentu, gdy zostanie wykonana operacja pull.

3. **Pull.** Operacja pull służy do pobierania zmian z zdalnego repozytorium i scalania ich z lokalnym repozytorium. Jest to połączenie dwóch operacji: fetch, który pobiera zmiany ze zdalnego repozytorium, oraz merge, który scala te zmiany z aktualnym stanem lokalnego repozytorium.

4. **Pull Request.** Pull request (w niektórych systemach znany jako merge request) jest funkcją oferowaną przez platformy hostingowe takie jak GitHub, GitLab czy Bitbucket. Umożliwia ona deweloperom proponowanie zmian w kodzie, które następnie mogą być przeglądane, dyskutowane i w końcu scalone z główną gałęzią projektu. Pull request jest kluczowym elementem współpracy i przeglądu kodu w wielu projektach open source oraz komercyjnych.

**Podstawowe pojęcia Git:**

1. **Repozytorium (Repository)** Repozytorium Git to podstawowa jednostka przechowywania kodu źródłowego. Jest to katalog, w którym Git przechowuje wszystkie metadane i obiekty bazy danych projektu. Repozytorium zawiera pełną historię wszystkich zmian dokonanych w projekcie oraz informacje o autorach tych zmian, co umożliwia śledzenie postępu prac i przywracanie starszych wersji kodu. Istnieją dwa typy repozytoriów w Git:
   a) **Lokalne Repozytorium.** Znajduje się na komputerze dewelopera i zawiera wszystkie pliki źródłowe oraz historię zmian w formie commitów. Tutaj deweloperzy dokonują zmian, tworzą nowe commity i zarządzają branchami.
   b) **Zdalne Repozytorium.** Jest to wersja repozytorium hostowana na serwerze (np. GitHub, GitLab, Bitbucket). Służy jako centralne miejsce, w którym zespół może współdzielić zmiany, a także jako backup. Zmiany są przesyłane do zdalnego repozytorium za pomocą operacji push i pobierane z niego za pomocą pull.

2. **Gałąź (Branch)** Gałąź w Git to lekki, ruchomy wskaźnik na commity. Pozwala ona na równoległe prowadzenie różnych linii rozwoju projektu. Gdy tworzona jest nową gałąź, Git tworzy nowy wskaźnik na aktualny commit. Następnie, po stworzeniu nowego commitu, wskaźnik gałęzi automatycznie przesuwa się do przodu, wskazując na najnowszy commit.
   - **Gałąź główna (Master/Main Branch).** Jest to domyślna gałąź w repozytorium, gdzie znajduje się "oficjalny"stan projektu.
   - **Gałęzie rozwojowe (Feature/Branches).** Są tworzone, by rozwijać nowe funkcjonalności, naprawiać błędy lub eksperymentować, bez wpływu na gałąź główną. Po zakończeniu pracy, zmiany z gałęzi rozwojowej mogą być scalone z gałęzią główną za pomocą operacji merge lub przez pull request.

   Wykorzystanie gałęzi umożliwia deweloperom pracę nad różnymi funkcjami jednocześnie, bez zakłócania stabilnej wersji produktu. Jest to szczególnie przydatne w zespołach, gdzie wielu programistów pracuje nad tym samym projektem.

Te podstawowe funkcje Git stanowią rdzeń zarządzania kodem źródłowym i są niezbędne dla efektywnej i bezpiecznej pracy zespołowej nad projektami programistycznymi.

### IV.1.7. Github i Github Actions

Github jest platformą internetową, która umożliwia hostowanie projektów informatycznych wykorzystujących system kontroli wersji Git. Github jest największą na świecie platformą tego typu. Github Actions to platforma do automatyzacji zadań, która umożliwia tworzenie niestandardowych procesów CI/CD bezpośrednio w repozytorium Github. Github Actions jest dostępny dla wszystkich repozytoriów publicznych i prywatnych, bez dodatkowych opłat. Github Actions jest zintegrowany z Github, co oznacza, że można łatwo tworzyć, uruchamiać i debugować procesy CI/CD bezpośrednio w repozytorium Github. Github Actions jest również zintegrowany z wieloma innymi narzędziami, takimi jak Kubernetes, Terraform, AWS, Azure, Google Cloud, Slack, Docker, itp. Github Actions jest dostępny jako usługa w chmurze lub jako oprogramowanie do instalacji w sieci lokalnej.

## IV.2. Instalacja Kubeflow

Instalacja Kubeflow składa się z kilku kroków:

1. Instalacja systemu operacyjnego Ubuntu Server - najlepiej Ubuntu 22.04, gdyż jest to rekomendowany system do Microk8s.

2. Instalacja microk8s używając wbudowanego w Ubuntu menadżera paczek snap. Trzeba wybrać odpowiednią wersję rekomendowaną przez Kubeflow,czyli wersja 1.26.

   ```
   sudo snap install microk8s --channel=1.26 --classic
   ```

3. Włączenia dodatków microk8s, które są wymagane do instalacji Kubeflow oraz Twingate.

   ```
   microk8s enable dns storage helm
   ```

4. Instalacja kustomize, który jest wymagana do instalacji Kubeflow. Następnie należy dodać go do PATH, czyli zmiennej środowiskowej, która zawiera listę katalogów, w których system operacyjny szuka plików wykonywalnych.

   ```
   sudo snap install kustomize
   echo 'export PATH=$PATH:/snap/bin' >> ~/.bashrc
   ```

5. Dodanie aliasu dla kubectl, czyli narzędzia do zarządzania klastrami Kubernetes.

   ```
   sudo snap alias microk8s.kubectl kubectl
   ```

6. Modyfikacja ustawień systemu max_user_instances i max_user_watches, aby zapobiec błędom podczas instalacji Kubeflow związanych z limitem zasobów.

```
sudo sh -c 'echo "fs.inotify.max_user_instances=1280" >> \
/etc/sysctl.conf' && sudo sysctl -p

sudo sh -c 'echo "fs.inotify.max_user_watches=655360" >> \
/etc/sysctl.conf' && sudo sysctl -p
```

7. Sklonowanie wcześniej przygotowanego repozytorium Kubeflow, które zawiera wszystkie pliki potrzebne do instalacji Kubeflow i przejście do katalogu z plikami.

```
git clone https://github.com/Matt-Szymczyk/kf-manifests
cd kf-manifests
```

8. Istalacja Kubeflow używając kustomize. Komenda ta może się wydawać skomplikowana, ale jest to tylko jedna linijka, która instaluje Kubeflow używając plików z katalogu kf-manifests, który został sklonowany w poprzednim kroku. Jest to pętla while, która sprawdza czy wszystkie zasoby zostały poprawnie utworzone. Jeżeli nie, to czeka 10 sekund i próbuje ponownie.

```
while ! kustomize build duchpuszczy | \
kubectl apply -f -; do echo "Retrying to apply resources"; sle
```

Utworzenie wszystkich zasobów może potrwać nawet kilkadziesiąt minut. Po tym czasie będzie możliwe połączenie lokalne z dashboardem.

```
kubectl port-forward svc/istio-ingressgateway -n istio-system
```

Po wykonaniu tej komendy będzie możliwe połączenie się z dashboardem Kubeflow pod adresem http://localhost:8080 z maszyny na której jest zainstalowany Kubeflow. W celu zalogowania się należy użyć adresu email oraz hasła, domyślnie jest to user@example.com oraz haslo 12341234. Potem można je zmienić.

Dostęp do dashboardu Kubeflow z zewnątrz klastra jest możliwy dzięki Twingate. Jest to rozwiązanie, które umożliwia bezpieczny dostęp do zasobów w chmurze, w sieci lokalnej i w aplikacjach SaaS. Twingate jest dostępne jako usługa w chmurze lub jako oprogramowanie do instalacji w sieci lokalnej. W tym przypadku wykorzystałem instalacje w klastrze Kubernetes. Twingate jest dostępne jako Helm Chart, który jest pakietem dla Kubernetesa. W celu instalacji Twingate należy wykonać następujące kroki:

1. Założyć darmowe konto na stronie https://www.twingate.com/

2. Utworzyć sieć w Twingate. W tym celu należy wybrać opcję "+ Remote Network"w panelu administracyjnym. Nadać nazwę sieci oraz wybrać opcję "On Premise"jako typ sieci.

3. Wejść w ustawenia sieci i utworzyć nowy "Connector". Potem wybrać typ connectora jako Helm Chart. Następnie trzeba podążać za instrukcjami na ekranie. Przykładowo:

```
helm repo add twingate https://twingate.github.io/helm-charts

helm repo update

helm upgrade
    --install twingate-connectortest twingate/connector
    -n default
    --set connector.network="duchpuszczy"
    --set connector.accessToken="xxxxxxxxxx"
    --set connector.refreshToken="xxxxxxxxxx"
```

   gdzie xxxxxxxxxx to tokeny, które można znaleźć w panelu administracyjnym w Twingate.

4. Po instalacji connecora należy sprawdzić jakie IP wewnętrzne w klastrze Kubernetes zostało przypisane do dashboardu Kubeflow. W tym celu należy wykonać następującą komendę:

```
kubectl get svc istio-ingressgateway -n istio-system
```

5. Następnie należy dodać nowy „Resource” w panelu administracyjnym. Należy wpisać adres IP. Potem w polu Alias możliwe jest wybranie dowolnej nazwy, która będzie używana do połączenia z dashboardem Kubeflow. Na przykład „kubeflow.ultra”

Po wykonaniu tych kroków będzie możliwe połączenie się z dashboardem Kubeflow pod adresem https://kubeflow.ultra z dowolnego miejsca na świecie pod warunkiem, że jest dostęp do internetu oraz zainstalowany jest klient Twingate.

## IV.3. Testy akceptacyjne

Aby mieć pewność, że platforma została poprawnie zainstalowana oraz narzędzia współpracują ze sobą wykorzystywane są testy akceptacyjne (z ang. User Acceptance Tests). Aby wykonać takie testy należy:

1. Utworzyć notatnik Jupyter z dostępem do Kubeflow Pipelines..
2. Uruchomić terminal w notatniku.
3. Sklonować repozytorium zawierające testy przygotowane dla dystrybucji Charmed Kubeflow.

```
git clone https://github.com/canonical/charmed-kubeflow-uats.git
```

4. Przejść do folderu z testami.

```
cd charmed-kubeflow-uats/tests
```

5. Zainstalować zależności python.

```
pip install -r requirements.txt
```

6. Uruchomić wszystkie testy za wyjątkiem takich które zawierają komponent MLFlow, gdyż nie jest on uwzględniony w implementacji.

```
pytest -k "not mlflow"
```

```
(base) jovyan@uats-0:~/charmed-kubeflow-uats/tests$ pytest -k 'not mlflow'
============================= test session starts ==============================
platform linux -- Python 3.8.10, pytest-7.4.4, pluggy-1.3.0 -- /opt/conda/bin/python3.8
cachedir: .pytest_cache
rootdir: /home/jovyan/charmed-kubeflow-uats/tests
configfile: pytest.ini
plugins: anyio-3.7.1
collected 9 items / 4 deselected / 5 selected

test_notebooks.py::test_notebook[katib-integration]
-------------------------------- live log call ---------------------------------
INFO     test_notebooks:test_notebooks.py:44 Running katib-integration.ipynb...
PASSED
test_notebooks.py::test_notebook[kfp-v1-integration]
-------------------------------- live log call ---------------------------------
INFO     test_notebooks:test_notebooks.py:44 Running kfp-v1-integration.ipynb...
PASSED
test_notebooks.py::test_notebook[kfp-v2-integration]
-------------------------------- live log call ---------------------------------
INFO     test_notebooks:test_notebooks.py:44 Running kfp-v2-integration.ipynb...
PASSED
test_notebooks.py::test_notebook[kserve-integration]
-------------------------------- live log call ---------------------------------
INFO     test_notebooks:test_notebooks.py:44 Running kserve-integration.ipynb...
PASSED
test_notebooks.py::test_notebook[training-integration]
-------------------------------- live log call ---------------------------------
INFO     test_notebooks:test_notebooks.py:44 Running training-integration.ipynb...
PASSED
```

**Rys. 11. Testy akceptacyjne**

Źródło: BAdanie własne

## IV.4. CI/CD w Kubeflow

Implelementacja CICD w ramach tej pracy dyplomowej składa się z kilku kroków:

1. Developer tworzy nowy branch w repozytorium Github.

2. Każdy push do repozytorium Github uruchamia skrypt CI, który sprawdza czy wszystkie pliki są poprawne składniowo oraz czy nie ma błędów w kodzie.

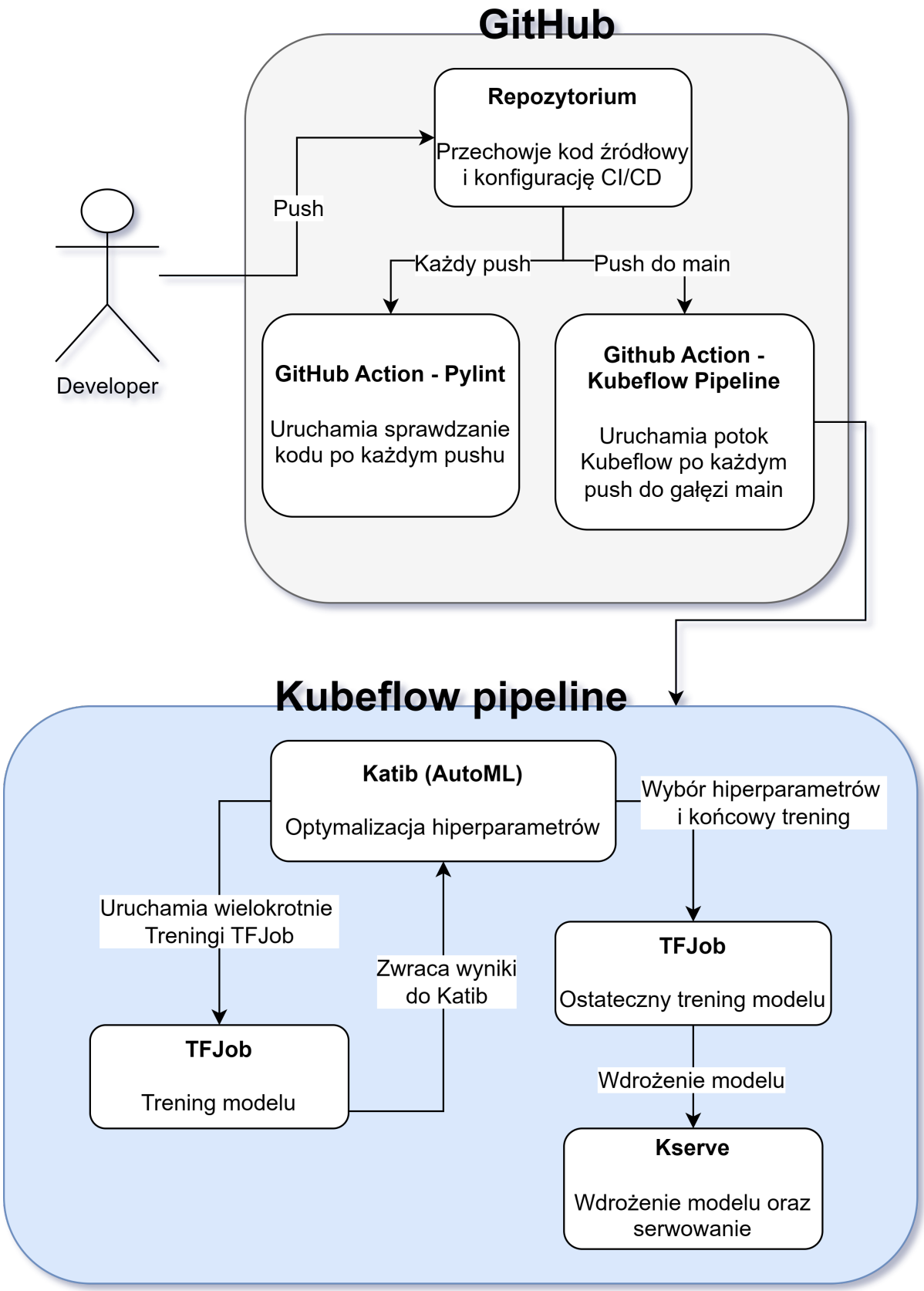
3. Po zakończonym procesie pisania kodu developer tworzy pull request do gałęzi main.

4. Po utworzeniu pull requesta również uruchamiany jest skrypt CI tak jak w punkcie 2.

5. Jeżeli skrypt CI zwróci status "Success"to pull request może zostać zaakceptowany oraz połączony z gałęzią main.

6. Po połączeniu z gałęzią main uruchamiany jest skrypt CD, który tworzy Kubeflow Pipeline.

Wynikiem końcowym Kubeflow Pipeline jest model uczenia maszynowego udostępniony w formie REST API dostępnego wewnątrz klastra Kubernetes.




**Rys. 12. Schemat CI/CD**

Źródło: Architecture, 2023, Architecture, Kubeflow.org, https://www.kubeflow.org/docs/started/architecture/ (dostęp: 10 grudnia 2023).

### IV.4.1. Implementacja Continous Integration

W celu implementacji CI wykorzystałem Github Actions. Pozwala ono na uruchamianie skryptów w chmurze Githuba. Podczas każdego pusha do repozytorium uruchamiany jest skrypt, który sprawdza czy wszystkie pliki są poprawne składniowo oraz czy nie ma błędów w kodzie. Jeżeli wszystko jest poprawne, to Github Actions zwraca status "Success"i pull request lub push jest akceptowany. Jeżeli wystąpi błąd, to Github Actions zwraca status "Failed"i pull request lub push jest odrzucony.

**Kod. 1. Skrypt Continuous Integration**

```yaml
name: Pylint

on: [push] # Wybranie zdarzenia, które uruchomi skrypt CI

jobs:
    build:
    runs-on: ubuntu-latest # Wybranie systemu operacyjnego
    strategy: # Definicja strategii uruchamiania
        matrix: # Uruchomienie równolegle, różne zmienne
        python-version: ["3.10", "3.11"] # Wersje Pythona
    steps: # Definicja kroków
    - uses: actions/checkout@v3 # Pobranie repozytorium
    - name: Instalacja Pythona ${{ matrix.python-version }} #
        uses: actions/setup-python@v3
        with:
        python-version: ${{ matrix.python-version }}
    - name: Instalacja zależności
        run: |
        python -m pip install --upgrade pip
        pip install pylint
    - name: Uruchomienie sprawdzania kodu
        run: |
        pylint $(git ls-files '*.py')
```

Źródło: Badania własne.

### IV.4.2. Implementacja Continius Delivery

Do implementacji CD również wykorzystałem Github Actions. Podczas każdego pusha do gałęzi main uruchamiane są skrypty.

**Kod. 2. Nazwa kodu źródłowego**

```
name: CICD

on: # Wybranie zdarzenia, które uruchomi workflow
push: # Wybranie zdarzenia push
branches: [ main ] # Wybranie gałęzi, która uruchomi workflow

jobs: # Definicja jobów
build:
runs-on: ubuntu-latest # Wybranie systemu operacyjnego

steps: #
- uses: actions/checkout@v2

- name: Instalacja Pythona
    uses: actions/setup-python@v2
    with:
    python-version: '3.11'

- name: Instalacja zależnosci
    run: |
    python -m pip install --upgrade pip
    if [ -f requirements.txt ];
    then pip install -r requirements.txt;
    fi

- name: Polaczenie z Twingate
    uses: twingate/github-action@main
    with:
    service-key: ${{ secrets.SERVICE_KEY_SECRET }}
                                $

- name: Uruchomienie skryptu Pythona tworzacego Kubeflow Pipeline
    run: |
    python kubeflow_action.py

```

Źródło: Badania własne

### IV.4.3. Implementacja Kubeflow Pipeline

Do przedstawienia działania Kubeflow Pipeline wykorzystałem przykład z oficjalnej dokumentacji Kubeflow[11]. Celem jego jest stworzenie systemu rozpoznającego ręcznie pisane cyfry. W tym celu użyty został zbiór danych MNIST[34]. Zbiór MNIST składa się z 70 000 obrazów cyfr o rozmiarze 28x28 pikseli. Każdy obraz jest oznaczony etykietą, która określa jaką cyfrę przedstawia. Jest podzielony na 60 000 obrazów treningowych oraz 10 000 obrazów testowych.

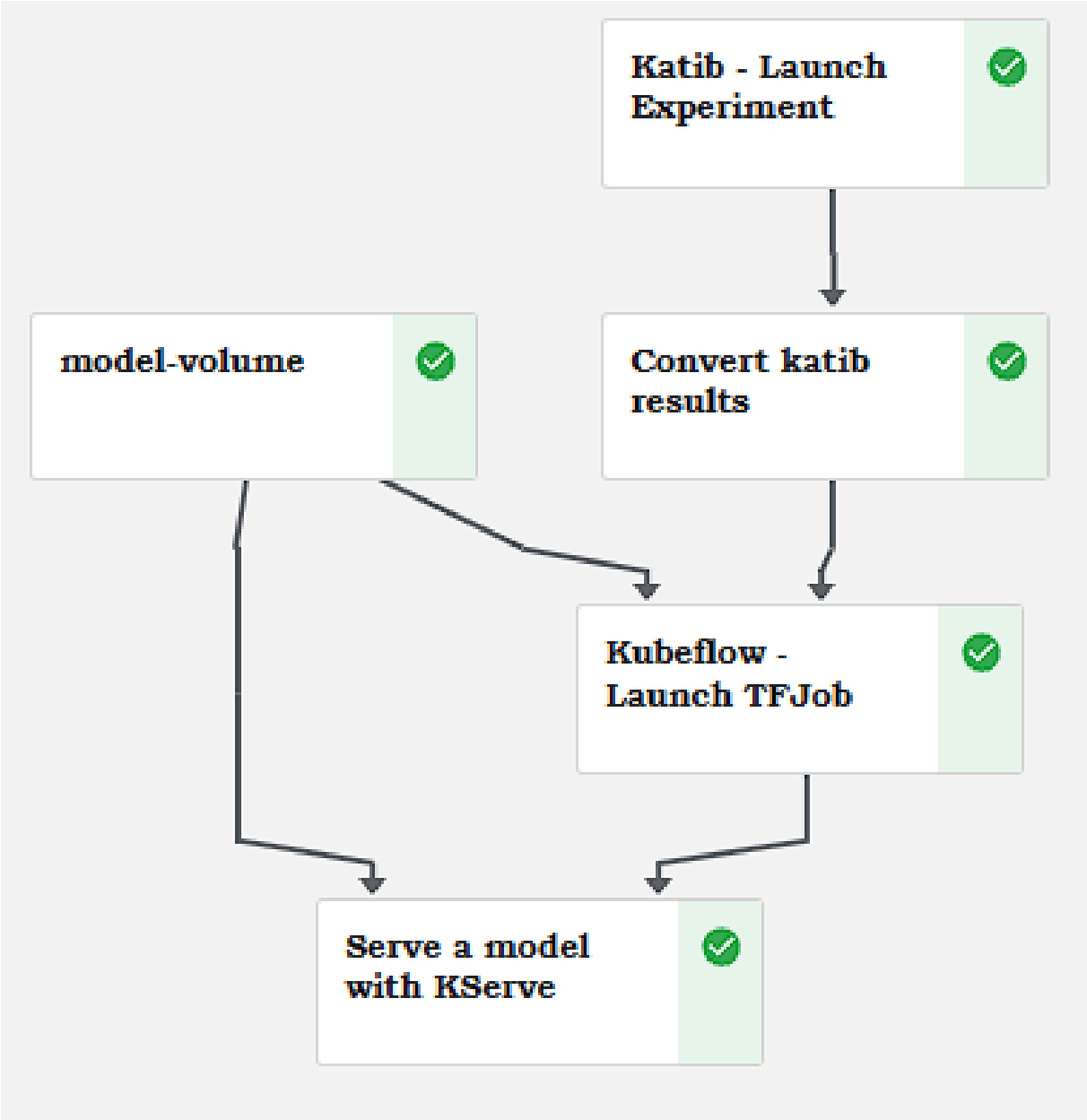
Pipeline przedstawiony na rysunku 13 składa się z następujących kroków

1. **Hiperparametryzacja z Katib.** Katib jest wykorzystywany do automatyzacji procesu doboru hiperparametrów. W tym przypadku, używa się algorytmu losowego ("random") do minimalizacji metryki "loss", eksplorując przestrzeń parametrów takich jak learning_rate i batch_size.

2. **Trening Modelu z TFJob.** Uruchamiany jest zadanie treningowe przy użyciu TFJob. TFJob pozwala na elastyczne skalowanie i zarządzanie treningiem w środowisku Kubernetes, wykorzystując określone hiperparametry.

3. **Eksportowanie Modelu.** Po zakończeniu treningu, model jest eksportowany voluem PVC (Persistent Volume Claim). Jest to zasób, który może być używany przez inne komponenty Kubeflow.

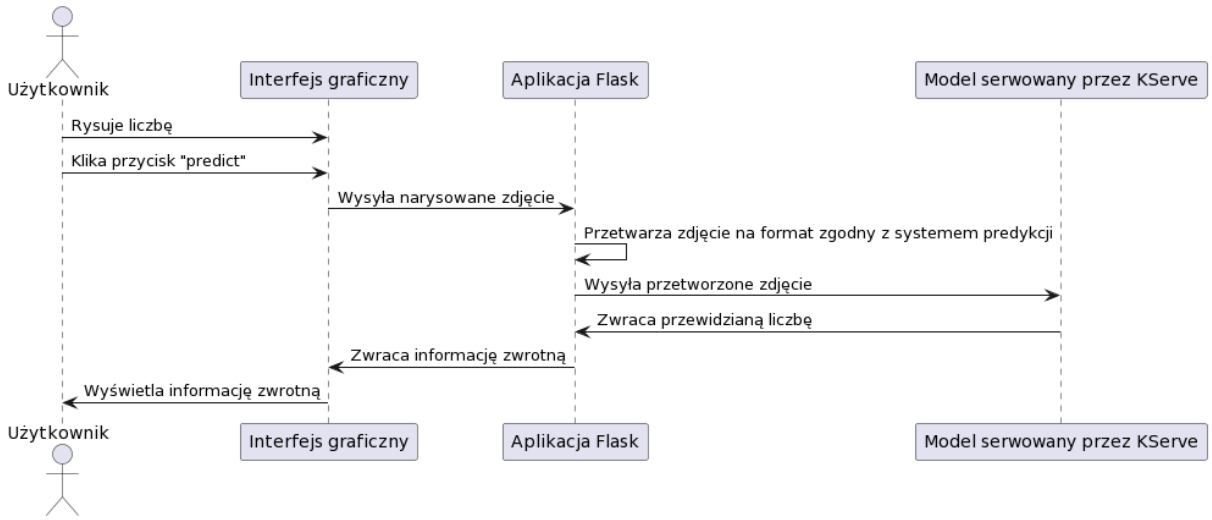
4. **Wdrożenie Modelu.** Model jest wdrażany przez Kserve jako usługa REST API.




**Rys. 13. Diagram działania CI/CD**

Źródło: Badanie własne

## IV.5. Implementacja Mikrousługi




**Rys. 14. Diagram przypadków użycia mikrousługi**

Źródło: Badanie własne

W celu użycia modelu stworzona została mikrousługa, której działanie przedstawione jest na rysunku 14. Zamienia ona narysowany przez użytkownika obraz na format zgodny z modelem, czyli 28x28 pixeli, białe cyfry na czarnym tle. Następnie wysyła obraz do modelu, który zwraca cyfrę rozpoznaną przez model.

Są trzy sposoby na jej wdrożenie:

1. Hostować na maszynie, która ma dostęp do klastra poprzez Twingate
2. Wystawić model serwowany przez Kserve poza klaster Kubernetes
3. Hostować bezpośrednio model w klastrze

W celach demonstracyjnych najlepszy jest sposób pierwszy, gdyż przydatny jest tryb debug oferowany przez Flask. Aby uruchomić mikrousługę należy wykonać następujące kroki:

1. Zainstalować Python 3.11
2. Zainstalować zależności
3. Sprawdzić adres IP usługi serwowania modelu w klastrze Kubernetes. W tym celu należy wykonać następującą komendę:

```
kubectl get svc -n kubeflow-user-example-com
```

4. Dodać adres IP do Twingate jako nowy zasób
5. Zmienić adres IP w pliku app.py na adres IP z poprzedniego kroku
6. Uruchomić mikrousługę za pomocą komendy

```
python app.py
```

7. W przeglądarce internetowej wejść na adres http://localhost:5000
8. Narysować cyfrę i kliknąć przycisk "Predict"
9. Przeglądarka powinna wyświetlić cyfrę rozpoznaną przez model
10. W celu zatrzymania mikrousługi należy użyć kombinacji klawiszy Ctrl+C

Przykładowe użycie mikrousługi przedstawia rysunek 13.




**Rys. 15. Przykładowe działanie mikrousługi**

Źródło: Badanie własne

## Podsumowanie

Niniejsza praca dyplomowa, skupiająca się na środowisku CI/CD dla modeli uczenia maszynowego, z sukcesem realizuje postawione cele. Przedstawiona analiza i implementacja wskazuje na istotność integracji platformy Kubeflow z narzędziami i technikami CI/CD dla osiągnięcia wydajności i niezawodności w procesie wdrażania modeli uczenia maszynowego.

Podstawowym osiągnięciem pracy jest dogłębne omówienie i implementacja środowiska CI/CD, skupiające się na wykorzystaniu platformy Kubeflow. Przedstawione rozwiązania odpowiadają na dynamicznie rozwijające się potrzeby w zakresie MLOps, oferując nowatorskie podejście do integracji narzędzi CI/CD z procesami uczenia maszynowego.

Porównanie uzyskanych wyników z publikacjami innych autorów ukazuje unikalność podejścia, zwłaszcza w kontekście wykorzystania platformy Kubeflow. Różnice te podkreślają innowacyjność pracy oraz jej wkład w rozwój narzędzi i technik CI/CD w obszarze uczenia maszynowego.

W ramach perspektyw rozwoju pracy zidentyfikowano cztery kluczowe obszary: implementacja certyfikatów TLS, obsługa GPU, monitoring zasobów klastra Kubernetes oraz implementacja wielu użytkowników. Te kierunki oferują znaczący potencjał dla dalszej optymalizacji i rozbudowy środowiska CI/CD, co może przyczynić się do zwiększenia efektywności i adaptacji rozwiązań w przyszłych projektach.

Podsumowując, praca ta stanowi cenny wkład w rozwój środowisk CI/CD dla modeli uczenia maszynowego, oferując praktyczne rozwiązania i wskazując kierunki przyszłych badań w tej dynamicznie rozwijającej się dziedzinie.

## Bibliografia


[1] Rodrigo Fernandes de Mello, Moacir Antonelli Ponti, 2018, Machine Learning A Practical Approach on the Statistical Learning Theory, „SpringerLink", 978-3-319-94989-5

[2] "What is MLflow? — MLflow 2.8.1 documentation", 2023, Mlflow.org, dostępny online: https://mlflow.org/docs/latest/introduction/index.html [dostęp: 3 grudnia 2023].

[3] Hapke H., Nelson C.: Building Machine Learning Pipelines. O'Reilly Media, 2020, ISBN 978-1-492-05319-4.

[4] van Merode H.: Continuous Integration (CI) and Continuous Delivery (CD): A Practical Guide to Designing and Developing Pipelines. Apress, 2023, ISBN 978-1-4842-9227-3.

[5] "Introduction to Kubeflow", 2021, Kubeflow.org, dostępny online: https://www.kubeflow.org/docs/started/introduction/ [dostęp: 10 grudnia 2023].

[6] "Build a pipeline", 2023, Google Cloud, dostępny online: https://cloud.google.com/vertex-ai/docs/pipelines/build-pipeline [dostęp: 10 grudnia 2023].

[7] "Managed MLflow | Databricks", Databricks, dostępny online: https://www.databricks.com/product/managed-mlflow [dostęp: 3 grudnia 2023].

[8] "Metaflow - a framework for real-life data science and ML", 2023, Metaflow.org, dostępny online: https://metaflow.org/ [dostęp: 3 grudnia 2023].

[9] "Introducing MLflow: an Open Source Machine Learning Platform", Databricks, dostępny online: https://www.databricks.com/blog/2018/06/05/introducing-mlflow-an-open-source-machine-learning-platform.html [dostęp: 3 grudnia 2023].

[10] "Why Metaflow", 2023, Metaflow Docs, dostępny online: https://docs.metaflow.org/introduction/why-metaflow [dostęp: 4 grudnia 2023].

[11] "Flyte documentation", 2022, Flyte.org, dostępny online: https://docs.flyte.org/en/latest/ [dostęp: 4 grudnia 2023].

[12] "MLReef Docs", 2022, Mlreef.com, dostępny online: https://docs.mlreef.com/ [dostęp: 4 grudnia 2023].

[13] "Overview of Seldon Core Components — seldon-core documentation", 2021, Seldon.io, dostępny online: https://docs.seldon.io/projects/seldon-core/en/latest/workflow/overview.html [dostęp: 4 grudnia 2023].

[14] "Sematic Features", 2023, Sematic.dev, dostępny online: https://www.sematic.dev/sematic-features [dostęp: 4 grudnia 2023].

[15] Slisenka C., 2022, "Orchestrating Data Pipelines at Lyft: comparing Flyte and Airflow", Medium, dostępny online: https://eng.lyft.com/orchestrating-data-pipelines-at-lyft-comparing-flyte-and-airflow- [dostęp: 4 grudnia 2023].

[16] "kubeflow/kubeflow: Machine Learning Toolkit for Kubernetes", 2023, GitHub, dostępny online: https://github.com/kubeflow/kubeflow [dostęp: 10 grudnia 2023].

[17] Karau H., Grant T., Lublinsky B., Liu R., Filonenko I.: Kubeflow for Machine Learning. O'Reilly Media, 2021, ISBN 978-1-492-05012-4.

[18] Patterson J., Katzenellenbogen M., Harris A.: Kubeflow Operations Guide. O'Reilly Media, 2021, ISBN 978-1-492-05327-9.

[19] Ibarra F., 2016, "Google takes Cloud Machine Learning service mainstream", Google Cloud Blog, dostępny online: https://cloud.google.com/blog/products/gcp/google-takes-cloud-machine-learning-service-mainstream [dostęp: 11 grudnia 2023].

[20] Stack M., 2017, "Introducing Kubeflow - A Composable, Portable, Scalable ML Stack Built for Kubernetes", Kubernetes Blog, dostępny online: https://kubernetes.io/blog/2017/12/introducing-kubeflow-composable/ [dostęp: 11 grudnia 2023].

[21] "Announcing Kubeflow 0.1", 2018, Kubernetes Blog, dostępny online: https://kubernetes.io/blog/2018/05/04/announcing-kubeflow-0.1/ [dostęp: 12 grudnia 2023].

[22] Lamkin T., 2020, "Kubeflow 1.0 - Cloud Native ML for Everyone", Kubeflow Blog, dostępny online: https://blog.kubeflow.org/releases/2020/03/02/kubeflow-1-0-cloud-native-ml-for-everyone.html [dostęp: 12 grudnia 2023].

[23] Lamkin T., 2022, "Kubeflow has applied to become a CNCF incubating project", Kubeflow Blog, dostępny online: https://blog.kubeflow.org/kubeflow-applied-cncf-incubating/ [dostęp: 12 grudnia 2023].

[24] "Kubeflow brings MLOps to the CNCF Incubator", 2023, CNCF Blog, dostępny online: https://www.cncf.io/blog/2023/07/25/kubeflow-brings-mlops-to-the-cncf-incubator/ [dostęp: 12 grudnia 2023].

[25] "October 2023: where we are with velocity of CNCF, LF, and top 30 open source projects", 2023, CNCF Blog, dostępny online: https://www.cncf.io/blog/2023/10/27/october-2023-where-we-are-with-velocity-of-cncf-lf-and-top-30- [dostęp: 12 grudnia 2023].

[26] "MicroK8s - Zero-ops Kubernetes for developers, edge and IoT", 2023, MicroK8s, dostępny online: https://microk8s.io/ [dostęp: 17 grudnia 2023].

[27] Laster B., O'Reilly Media, Inc., January 2022, "Leveraging Kustomize for Kubernetes Manifests".

[28] "cvdfoundation/mnist: The MNIST database of handwritten digits", 2023, GitHub, dostępny online: https://github.com/cvdfoundation/mnist?tab=readme-ov-file [dostęp: 27 grudnia 2023].

[29] Chacon S. i Straub B., Apress, 2014, "Pro Git"

[30] Twingate: It's time to ditch your VPN, 2024, Twingate.com, https://www.twingate.com/ (dostęp: 2 stycznia 2024).

## Spis rysunków

Rys. 1. Cykl życia modelu uczenia maszynowego ..... 6
Rys. 2. Panel Nawigacyjny(Dashboard) Kubeflow ..... 15
Rys. 3. Panel Kubeflow Notebooks ..... 17
Rys. 4. Panel tworzenia Kubeflow Notebooks ..... 17
Rys. 5. Przykładowy przepływ pracy w Kubeflow Pipelines ..... 18
Rys. 6. Przykład eksperymentu w Katib ..... 19
Rys. 7. Przykład wdrożenia modelu w Kubeflow ..... 20
Rys. 8. Przepływ pracy w Kubeflow ..... 22
Rys. 9. Architentura implementacji ..... 23
Rys. 10.Przykład użycia Kustomize ..... 27
Rys. 11.Testy akceptacyjne ..... 33
Rys. 12.Schemat CI/CD ..... 35
Rys. 13.Diagram działania CI/CD ..... 39
Rys. 14.Diagram przypadków użycia mikrousługi ..... 40
Rys. 15.Przykładowe działanie mikrousługi ..... 41

## Spis tabel

## Załączniki

1. Płyta CD/DVD zawierająca:
    a) Prezentację wyników pracy dyplomowej
    b) Kody źródłowe oprogramowania
    c) Biblioteki programowe niezbędne do zbudowania i uruchomienia oprogramowania